

```

In [ ]: # =====
# Multimodal VGG16 with Concatenation Fusion
# - Uses 7-Fold CV ≈ 70% Train / 15% Val / 15% Test Overall
# - Implements VGG16 as the image backbone within the established multimodal pipeline
# - MODIFIED: Swapped DINOv3 with VGG16, adjusted feature extraction and fine-tuning
# =====

import os, time, json, warnings
from copy import deepcopy
from collections import defaultdict
from itertools import cycle
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms, models
from PIL import Image

from sklearn.model_selection import StratifiedKFold, StratifiedShuffleSplit
from sklearn.metrics import (
    classification_report, accuracy_score, cohen_kappa_score,
    matthews_corrcoef, roc_auc_score, confusion_matrix, roc_curve, auc
)
from sklearn.preprocessing import label_binarize

from torch.amp import GradScaler, autocast

warnings.filterwarnings('ignore')
torch.backends.cudnn.benchmark = True

# Optional FLOPs (best effort)
try:
    from thop import profile
    HAVE_THOP = True
except Exception:
    HAVE_THOP = False

# Optional plotting libs
try:
    import matplotlib.pyplot as plt
    import seaborn as sns
    HAVE_PLOTTING = True
except ImportError:
    HAVE_PLOTTING = False

# -----
# 1) CONFIG – UPDATE THESE TWO PATHS
# -----
class Config:
    DATA_DIR = r"D:\AIUB\Neural\Synthetic_Skin_Disease_Dataset" # <-- update to

```

```

ORIGINAL_METADATA_CSV = r"D:\AIUB\Neural\complete_metadata_m.csv" # <-- update

# === MODEL SWAP: VGG16 ===
RESULTS_DIR = "metadata_Multimodal_VGG16_Concat_KFold" # <-- New directory for

IMG_SIZE = 224
BATCH_SIZE = 128
LEARNING_RATE = 1e-4      # A good starting point for VGG16
WEIGHT_DECAY = 1e-4
EPOCHS = 200
NUM_CLASSES = 5
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
USE_AMP = True
NUM_WORKERS = 0
PATIENCE = 20
WARMUP_EPOCHS = 10
LABEL_SMOOTHING = 0.1
PARTIALLY_UNFREEZE = True # Fine-tuning the last block of VGG16 is recommended

# K-Fold settings for ~70/15/15 overall
N_SPLITS = 7
VAL_WITHIN_REMAINDER = 0.15 / (1.0 - 1.0/7.0) # ≈ 0.175

# Metadata settings
META_DROPOUT = 0.3
META_HIDDEN_DIM = 128
META_OUT_DIM = 256

# Fairness: minimum subgroup size for stable metrics
MIN_SUPPORT = 15

config = Config()
os.makedirs(config.RESULTS_DIR, exist_ok=True)
print("Device:", config.DEVICE)
print(f"Model: VGG16 with Concatenation Fusion")
print(f"K-Fold: N_SPLITS={config.N_SPLITS} (~Test {100/config.N_SPLITS:.1f}%), Val_
if not HAVE_PLOTTING:
    print("Warning: matplotlib and/or seaborn not found. Plots will be skipped.")

# -----
# 2) CSV STANDARDIZATION
# -----
def standardize_metadata_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Standardizes metadata column names and creates necessary fields."""
    df = df.copy()
    if 'filepath' in df.columns:
        df['filepath'] = df['filepath'].astype(str).str.replace('\\', '/', regex=False)
    if 'filename' not in df.columns:
        if 'filepath' in df.columns:
            df['filename'] = df['filepath'].apply(lambda p: os.path.basename(str(p)))
        else:
            raise ValueError("CSV must include 'filepath' or 'filename'.")
    if 'label' not in df.columns and 'filepath' in df.columns:
        df['label'] = df['filepath'].str.split('/').str[1]
    df['label'] = df['label'].astype(str)

```

```

if 'age_group' not in df.columns:
    if 'age' in df.columns:
        age_num = pd.to_numeric(df['age'], errors='coerce')
        bins = [-1, 17, 29, 44, 64, 200]
        names = ['child', 'young_adult', 'adult', 'middle_aged', 'senior']
        df['age_group'] = pd.cut(age_num, bins=bins, labels=names).astype(str).
    else:
        df['age_group'] = 'unknown'
if 'gender' not in df.columns:
    if 'dominant_gender' in df.columns:
        g = df['dominant_gender'].astype(str).str.strip().str.lower()
        df['gender'] = g.map({'man': 'male', 'male': 'male', 'woman': 'female', 'fema
    else:
        df['gender'] = 'unknown'
if 'skin_tone' not in df.columns:
    if 'dominant_race' in df.columns:
        df['skin_tone'] = df['dominant_race'].astype(str).str.strip().str.lower
    else:
        df['skin_tone'] = 'unknown'
required = ['filename', 'label', 'age_group', 'gender', 'skin_tone']
if any(c not in df.columns for c in required):
    raise ValueError(f"Missing required columns after standardization.")
return df

# -----
# 3) DATASET & VOCAB
# -----
class GPUGAugOnlyNorm(nn.Module):
    """Module for GPU-side normalization."""
    def __init__(self):
        super().__init__()
        self.normalize = transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.22
@torch.no_grad()
    def forward(self, x): return self.normalize(x)

class MetaVocab:
    """Creates and manages vocabularies for metadata fields."""
    def __init__(self, df_train, cols=('age_group', 'gender', 'skin_tone')):
        self.cols = cols
        self.unk_token = "<UNK>"
        self.vocabs = {}
        for c in cols:
            uniq = sorted([self.unk_token] + sorted(df_train[c].dropna().astype(str)
            self.vocabs[c] = {v:i for i,v in enumerate(uniq)}
        self.sizes = {c: len(self.vocabs[c]) for c in cols}
    def encode_row(self, row):
        out = {}
        for c in self.cols:
            v = str(row[c]) if pd.notnull(row[c]) else self.unk_token
            out[c] = self.vocabs[c].get(v, self.vocabs[c][self.unk_token])
        return out

class MultimodalDermDataset(Dataset):
    """Dataset class for images and metadata, with optional preloading."""
    def __init__(self, config, df, class_to_idx, meta_vocab, transform=None):

```

```

self.config = config
self.df = df.reset_index(drop=True)
self.class_to_idx = class_to_idx
self.transform = transform
self.meta_vocab = meta_vocab
self.image_paths, self.labels, self.meta = [], [], []
use_filepath = 'filepath' in self.df.columns
for _, row in self.df.iterrows():
    if use_filepath and pd.notnull(row.get('filepath', None)):
        raw = str(row['filepath'])
        p = raw if os.path.isabs(raw) else os.path.join(config.DATA_DIR, raw)
    else:
        p = os.path.join(config.DATA_DIR, str(row['split']), str(row['label']))
    self.image_paths.append(p)
    self.labels.append(self.class_to_idx[str(row['label'])])
    self.meta.append(self.meta_vocab.encode_row(row))
self.images = []
for p in self.image_paths:
    with Image.open(p).convert("RGB") as im:
        self.images.append(im.copy())

def __len__(self): return len(self.labels)
def __getitem__(self, idx):
    image = self.images[idx]
    if self.transform: image = self.transform(image)
    return image, self.labels[idx], self.meta[idx]

# -----
# 4) MODEL (VGG16 + Metadata Concatenation)
# -----
class MetadataEncoder(nn.Module):
    """Encodes categorical metadata into a dense feature vector."""
    def __init__(self, meta_vocab, emb_dims=None, hidden=128, out_dim=256, dropout=0.5):
        super().__init__()
        sizes = meta_vocab.sizes
        if emb_dims is None:
            emb_dims = {
                'age_group': min(16, max(4, sizes['age_group'])),
                'gender': min(8, max(3, sizes['gender'])),
                'skin_tone': min(16, max(4, sizes['skin_tone'])),
            }
        self.emb_age = nn.Embedding(sizes['age_group'], emb_dims['age_group'])
        self.emb_gen = nn.Embedding(sizes['gender'], emb_dims['gender'])
        self.emb_tone = nn.Embedding(sizes['skin_tone'], emb_dims['skin_tone'])
        concat_dim = sum(emb_dims.values())
        self.mlp = nn.Sequential(
            nn.Linear(concat_dim, hidden), nn.GELU(), nn.Dropout(dropout),
            nn.Linear(hidden, out_dim), nn.GELU()
        )
    def forward(self, meta_idx):
        a = self.emb_age(meta_idx['age_group'])
        g = self.emb_gen(meta_idx['gender'])
        s = self.emb_tone(meta_idx['skin_tone'])
        return self.mlp(torch.cat([a, g, s], dim=-1))

```

```

class VGG16MultimodalConcat(nn.Module):
    """
    A multimodal model using a pretrained VGG16 backbone.
    Features from the image backbone's convolutional base are concatenated
    with features from the metadata encoder and passed to a final classifier.
    """
    def __init__(self, num_classes, meta_vocab, meta_hidden_dim, meta_out_dim, meta_dropout):
        super().__init__()
        # Image backbone (VGG16)
        self.backbone = models.vgg16(weights=models.VGG16_Weights.IMAGENET1K_V1)

        # Freeze all backbone parameters initially
        for param in self.backbone.parameters():
            param.requires_grad = False

        # Unfreeze the final convolutional block if configured
        if config.PARTIALLY_UNFREEZE:
            print("Unfreezing the last VGG16 conv block for fine-tuning.")
            # Unfreeze layers from the 24th layer onwards (last conv block)
            for param in list(self.backbone.features.parameters())[-5:]:
                param.requires_grad = True

        # The feature dimension from VGG16's convolutional base
        # VGG16 with 224x224 input produces a 512x7x7 feature map
        image_feature_dim = 512 * 7 * 7

        # Metadata encoder
        self.meta_encoder = MetadataEncoder(meta_vocab,
                                            hidden=meta_hidden_dim,
                                            out_dim=meta_out_dim,
                                            dropout=meta_dropout)

        # Fusion and Classifier Head
        self.classifier = nn.Sequential(
            nn.Linear(image_feature_dim + meta_out_dim, 1024),
            nn.BatchNorm1d(1024),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(1024, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, num_classes)
        )

    def forward(self, pixel_values, meta_idx):
        # Image stream
        # Pass through the convolutional layers
        img_features = self.backbone.features(pixel_values)
        # Flatten the features
        img_features = torch.flatten(img_features, 1)

        # Metadata stream
        meta_features = self.meta_encoder(meta_idx)

        # Fusion by simple concatenation

```

```

        combined_features = torch.cat((img_features, meta_features), dim=1)

        # Classification
        output = self.classifier(combined_features)
        return output

# -----
# 5) TRAIN / EVAL HELPERS
# -----
def _to_device_meta(batch_meta, device):
    if isinstance(batch_meta, dict):
        ag = torch.as_tensor(batch_meta['age_group'], dtype=torch.long, device=device)
        ge = torch.as_tensor(batch_meta['gender'], dtype=torch.long, device=device)
        st = torch.as_tensor(batch_meta['skin_tone'], dtype=torch.long, device=device)
        return {'age_group': ag, 'gender': ge, 'skin_tone': st}
    else:
        ag = torch.tensor([m['age_group'] for m in batch_meta], dtype=torch.long, device=device)
        ge = torch.tensor([m['gender'] for m in batch_meta], dtype=torch.long, device=device)
        st = torch.tensor([m['skin_tone'] for m in batch_meta], dtype=torch.long, device=device)
        return {'age_group': ag, 'gender': ge, 'skin_tone': st}

def make_dataloaders_from_df(config, df_std):
    """Creates dataloaders for Train, Val, and Test splits from a DataFrame."""
    classes = sorted(df_std[df_std['split']=='Train']['label'].astype(str).unique())
    class_to_idx = {c:i for i,c in enumerate(classes)}
    config.NUM_CLASSES = len(classes)
    meta_vocab = MetaVocab(df_std[df_std['split']=='Train'])

    train_tfm = transforms.Compose([
        transforms.RandomResizedCrop(config.IMG_SIZE, scale=(0.8, 1.0)),
        transforms.RandomHorizontalFlip(),
        transforms.RandomApply([transforms.ColorJitter(0.4,0.4,0.2,0.1)], p=0.8),
        transforms.RandomRotation(15),
        transforms.ToTensor(),
    ])
    eval_tfm = transforms.Compose([
        transforms.Resize((config.IMG_SIZE, config.IMG_SIZE)),
        transforms.ToTensor(),
    ])
    dsets = {s: MultimodalDermDataset(config, df_std[df_std['split']==s], class_to_idx,
                                     for s, t in [('Train', train_tfm), ('Val', eval_tfm), ('Test', eval_tfm)])
    dls = {s: DataLoader(dsets[s], batch_size=config.BATCH_SIZE, shuffle=(s=='Train'),
                       num_workers=config.NUM_WORKERS, pin_memory=True) for s in dsets}
    return dls, {k: len(v) for k,v in dsets.items()}, classes, meta_vocab

def plot_and_save_history(hist, fold_id, results_dir):
    """Plots training and validation loss and accuracy and saves them to a file."""
    if not HAVE_PLOTTING: return

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 6))
    epochs = range(1, len(hist['train_loss']) + 1)
    ax1.plot(epochs, hist['train_loss'], 'bo-', label='Training loss')
    ax1.plot(epochs, hist['val_loss'], 'ro-', label='Validation loss')
    ax1.set_title(f'Training and Validation Loss (Fold {fold_id})'); ax1.set_xlabel('Epoch')
    ax1.legend(); ax1.grid(True)

```

```

ax2.plot(epochs, hist['train_acc'], 'bo-', label='Training accuracy')
ax2.plot(epochs, hist['val_acc'], 'ro-', label='Validation accuracy')
ax2.set_title(f'Training and Validation Accuracy (Fold {fold_id})'); ax2.set_xlabel(epochs)
ax2.legend(); ax2.grid(True)
plt.tight_layout()
history_path = os.path.join(results_dir, f'training_history_fold_{fold_id}.png')
plt.savefig(history_path, dpi=300); plt.show(); plt.close()
print(f"Saved training history plot to: {history_path}")

def train_model(model, dls, sizes, num_epochs, fold_id, class_weights):
    start = time.time()
    best = deepcopy(model.state_dict())
    best_val_loss = float('inf')
    patience = 0
    history = defaultdict(list)
    scaler = GradScaler(enabled=config.USE_AMP)
    gpu_norm = GPUAugOnlyNorm().to(config.DEVICE)
    optimizer = optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()),
                                lr=config.LEARNING_RATE, weight_decay=config.WEIGHT_DECAY)

    def lr_lambda(ep):
        if ep < config.WARMUP_EPOCHS: return float(ep+1)/config.WARMUP_EPOCHS
        return max(0.0, 0.5 * (1.0 + np.cos(np.pi * (ep - config.WARMUP_EPOCHS) / (num_epochs - config.WARMUP_EPOCHS))))
    scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

    criterion = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=config.LABEL_SMOOTHING)
    print(f"Using weighted CrossEntropyLoss for training.")

    best_model_path = os.path.join(config.RESULTS_DIR, f'best_model_fold_{fold_id}.pth')

    print(f"\nSTART TRAINING (VGG16 + Concat) for {num_epochs} epochs...")
    for epoch in range(num_epochs):
        for phase in ['Train', 'Val']:
            model.train(phase == 'Train')
            running_loss, running_corrects = 0.0, 0
            for images, labels, metas in dls[phase]:
                images = gpu_norm(images.to(config.DEVICE))
                labels = labels.to(config.DEVICE, dtype=torch.long)
                meta_idx = _to_device_meta(metas, config.DEVICE)
                optimizer.zero_grad(set_to_none=True)
                with autocast(device_type=config.DEVICE.split(':')[0], enabled=config.USE_AMP):
                    logits = model(pixel_values=images, meta_idx=meta_idx)
                    loss = criterion(logits, labels)
                if phase == 'Train':
                    scaler.scale(loss).backward(); scaler.step(optimizer); scaler.update()
                running_loss += loss.item() * images.size(0)
                running_corrects += (logits.argmax(1) == labels).sum().item()

            epoch_loss = running_loss / sizes[phase]
            epoch_acc = running_corrects / sizes[phase]
            history[f'{phase.lower()}_loss'].append(epoch_loss)
            history[f'{phase.lower()}_acc'].append(epoch_acc)

            if phase == 'Val':
                if epoch_loss < best_val_loss:
                    best_val_loss = epoch_loss; best = deepcopy(model.state_dict())
                    torch.save(model.state_dict(), best_model_path)

```

```

        else:
            patience += 1
        scheduler.step()
        print(f"Epoch {epoch+1}/{num_epochs} | Train Loss: {history['train_loss'][-1]:.4f} Val Loss: {history['val_loss'][-1]:.4f} Acc: {history['val_acc'][-1]:.4f}")
        if patience >= config.PATIENCE:
            print("Early stopping triggered."); break

    print(f"Training complete in {(time.time()-start)/60.0:.1f} minutes. Best Val Loss: {best_val_loss:.4f}")
    print(f"Best model for fold saved to: {best_model_path}")
    model.load_state_dict(best)
    plot_and_save_history(history, fold_id, config.RESULTS_DIR)
    return model, history

# -----
# 6) EVALUATION & FAIRNESS METRICS
# -----
def _print_section(title):
    print("\n" + "="*80); print(title.upper()); print("="*80 + "\n")

def _group_fairness_metrics(y_true, y_pred, groups, num_classes, min_support=None):
    uniq_groups = sorted(pd.Series(groups).astype(str).unique())
    per_group_acc, per_group_tprs, counts = {}, {}, {}
    y_true, y_pred = np.asarray(y_true), np.asarray(y_pred)
    for g in uniq_groups:
        mask = (np.asarray(groups) == g); counts[g] = int(mask.sum())
        yt, yp = y_true[mask], y_pred[mask]
        per_group_acc[g] = (yt == yp).mean() if len(yt) > 0 else np.nan
        tprs = [( (yt == c) & (yp == c)).sum() / ((yt == c).sum() or 1) ) for c in range(num_classes)]
        per_group_tprs[g] = np.array(tprs, dtype=float)
    filt_keys = [g for g in uniq_groups if counts[g] >= min_support] if min_support else uniq_groups
    wga = float(np.nanmin([per_group_acc.get(g, np.nan) for g in filt_keys])) if filt_keys else None
    max_gap, worst_pair = 0.0, (None, None)
    for i in range(len(filt_keys)):
        for j in range(i + 1, len(filt_keys)):
            gi, gj = filt_keys[i], filt_keys[j]
            a, b = per_group_tprs[gi], per_group_tprs[gj]
            mask = ~(np.isnan(a) | np.isnan(b))
            if mask.sum() > 0:
                gap = float(np.nanmean(np.abs(a[mask] - b[mask])))
                if gap > max_gap: max_gap, worst_pair = gap, (gi, gj)
    return {'wga': wga, 'eod': max_gap, 'worst_pair': worst_pair, 'keys_used': filt_keys}

def plot_and_save_roc_curves(y_true_bin, y_prob, class_names, fold_id, results_dir):
    if not HAVE_PLOTTING: return
    n_classes = len(class_names)
    fpr, tpr, roc_auc = dict(), dict(), dict()
    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_prob[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])
    all_fpr = np.unique(np.concatenate([fpr[i] for i in range(n_classes)]))
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(n_classes):
        mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])
    mean_tpr /= n_classes
    fpr["macro"], tpr["macro"], roc_auc["macro"] = all_fpr, mean_tpr, auc(all_fpr, mean_tpr)
    plt.figure(figsize=(12, 10))

```

```

colors = cycle(['aqua', 'darkorange', 'cornflowerblue', 'green', 'red', 'purple'])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2, label=f'ROC of {class_names[i]}')
plt.plot(fpr["macro"], tpr["macro"], label=f'Macro-average ROC (AUC = {roc_auc}')
plt.plot([0, 1], [0, 1], 'k--', lw=2); plt.xlim([0.0, 1.0]); plt.ylim([0.0, 1.0])
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate'); plt.title(
plt.legend(loc="lower right"); plt.grid(True)
roc_path = os.path.join(results_dir, f'roc_curves_fold_{fold_id}.png')
plt.savefig(roc_path, dpi=300); plt.show(); plt.close()
print(f"Saved ROC curves plot to: {roc_path}")

def plot_and_save_scalar_metrics(metrics_dict, fold_id, results_dir):
    if not HAVE_PLOTTING: return
    names, values = list(metrics_dict.keys()), list(metrics_dict.values())
    plt.figure(figsize=(10, 6))
    bars = sns.barplot(x=names, y=values, palette="viridis")
    plt.title(f'Overall Scalar Metrics Summary - Fold {fold_id}'); plt.ylabel('Score')
    for bar in bars.patches:
        plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.01, f'{bar.get_y():.2f}')
    scalar_path = os.path.join(results_dir, f'scalar_metrics_fold_{fold_id}.png')
    plt.savefig(scalar_path, dpi=300); plt.show(); plt.close()
    print(f"Saved scalar metrics plot to: {scalar_path}")

def evaluate_and_visualize(model, dl_test, class_names, df_fold, fold_id):
    model.eval()
    gpu_norm = GPUAugOnlyNorm().to(config.DEVICE)
    y_true, y_pred, y_prob = [], [], []
    with torch.no_grad():
        for images, labels, metas in dl_test:
            images = gpu_norm(images.to(config.DEVICE))
            meta_idx = _to_device_meta(metas, config.DEVICE)
            logits = model(pixel_values=images, meta_idx=meta_idx)
            y_true.extend(labels.numpy())
            y_pred.extend(logits.argmax(1).cpu().numpy())
            y_prob.extend(torch.softmax(logits, dim=1).cpu().numpy())
    y_true, y_pred, y_prob = np.array(y_true), np.array(y_pred), np.array(y_prob)

    _print_section(f"OVERALL PERFORMANCE (FOLD {fold_id})")
    print(classification_report(y_true, y_pred, target_names=class_names, zero_divi

    acc = accuracy_score(y_true, y_pred)
    kappa = cohen_kappa_score(y_true, y_pred)
    mcc = matthews_corrcoef(y_true, y_pred)
    try:
        y_true_bin = label_binarize(y_true, classes=np.arange(len(class_names)))
        roc_auc_macro = roc_auc_score(y_true_bin, y_prob, average='macro', multi_cl
    except Exception: y_true_bin, roc_auc_macro = None, float('nan')
    print(f"\n--- Overall Scalar Metrics ---\nAccuracy: {acc:.4f}\nMacro ROC AUC: {

    if HAVE_PLOTTING:
        cm = confusion_matrix(y_true, y_pred, labels=np.arange(len(class_names)))
        cm_norm = np.nan_to_num(cm.astype('float') / cm.sum(axis=1)[:, np.newaxis])
        plt.figure(figsize=(12, 10))
        sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues', xticklabels=class
        plt.title(f'Normalized Confusion Matrix (Fold {fold_id})'); plt.ylabel('True
        cm_path = os.path.join(config.RESULTS_DIR, f'confusion_matrix_fold_{fold_id}

```

```

plt.savefig(cm_path, dpi=300); plt.show(); plt.close()
print(f"Saved normalized confusion matrix to: {cm_path}")
if y_true_bin is not None: plot_and_save_roc_curves(y_true_bin, y_prob, cla
plot_and_save_scalar_metrics({'Accuracy': acc, 'Macro AUC': roc_auc_macro,

df_test = df_fold[df_fold['split'] == 'Test'].reset_index(drop=True)
df_test = df_test.iloc[:len(y_true)] # Align rows just in case
age_disp = df_test['age_group'].map({'child': '0-17', 'young_adult': '18-59', '

full_labels = np.arange(len(class_names))

for group_name, series in [('GENDER', df_test['gender']), ('AGE', age_disp), ('
    _print_section(f"PERFORMANCE BY {group_name}")
    for val in sorted(series.unique()):
        if val == 'unknown': continue
        mask = (series == val).values
        if mask.sum() > 0:
            print(f"--- Subgroup: {val} ---\n")
            print(classification_report(y_true[mask], y_pred[mask],
                                        target_names=class_names,
                                        labels=full_labels,
                                        zero_division=0))

groups_composite = (age_disp.astype(str) + "|" + df_test['gender'].astype(str)
fair_all = _group_fairness_metrics(y_true, y_pred, groups_composite, len(class_
fair_filt = _group_fairness_metrics(y_true, y_pred, groups_composite, len(class
print("\n" + "="*80 + f"\nFAIRNESS SUMMARY (ALL GROUPS) | WGA: {fair_all['wga']}
print("\n" + "="*80 + f"\nFAIRNESS SUMMARY (FILTERED, n>={config.MIN_SUPPORT})

def acc_counts(s, yt, yp): return {str(g): ((yt[s==g] == yp[s==g]).sum(), (s==g
pooled_info = {'gender': acc_counts(df_test['gender'], y_true, y_pred), 'age':

return {'accuracy': acc, 'roc_auc_macro': roc_auc_macro, 'kappa': kappa, 'mcc':

# -----
# 7) K-FOLD CROSS-VALIDATION ORCHESTRATION
# -----
def build_kfold_splits(df_std, n_splits, val_within_remainder, seed=42):
    df, y = df_std.copy().reset_index(drop=True), df_std['label'].astype(str).value
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
    folds = []
    for i, (trainval_idx, test_idx) in enumerate(skf.split(np.arange(len(df)), y),
        sss2 = StratifiedShuffleSplit(n_splits=1, test_size=val_within_remainder, r
        tr_rel, val_rel = next(sss2.split(trainval_idx, y[trainval_idx]))
        dfi = df.copy(); dfi['split'] = 'Ignore'
        dfi.loc[trainval_idx[tr_rel], 'split'], dfi.loc[trainval_idx[val_rel], 'spl
        folds.append(dfi[dfi['split'] != 'Ignore'])
    return folds

def run_one_fold(fold_df, fold_id, config):
    print("\n" + "="*88 + f"\n### FOLD {fold_id} ###\n" + "="*88)
    dls, sizes, class_names, meta_vocab = make_data loaders_from_df(config, fold_df)

    # Calculate class weights for the current fold's training data
    train_labels = fold_df[fold_df['split']=='Train']['label']

```

```

class_counts = train_labels.value_counts().sort_index().reindex(class_names, fill_value=0)
weights = 1.0 / (class_counts + 1e-6)
class_weights = torch.tensor(weights.values, dtype=torch.float).to(config.DEVICE)
class_weights = class_weights / class_weights.sum() * len(class_names)

# === MODEL SWAP: VGG16 ===
model = VGG16MultimodalConcat(
    num_classes=len(class_names),
    meta_vocab=meta_vocab,
    meta_hidden_dim=config.META_HIDDEN_DIM,
    meta_out_dim=config.META_OUT_DIM,
    meta_dropout=config.META_DROPOUT
).to(config.DEVICE)

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
total_params = sum(p.numel() for p in model.parameters())
print(f"Total params: {total_params/1e6:.2f}M | Trainable params: {trainable_params/1e6:.2f}M")

model, _ = train_model(model, dls, sizes, num_epochs=config.EPOCHS, fold_id=fold_id)
return evaluate_and_visualize(model, dls['Test'], class_names, fold_df, fold_id)

def kfold_cross_val(df_std, config):
    folds = build_kfold_splits(df_std, config.N_SPLITS, config.VAL_WITHIN_REMAINDER)
    per_fold = [run_one_fold(fold, i + 1, config) for i, fold in enumerate(folds)]
    perf = pd.DataFrame(per_fold)

    def mean_std(col): return (perf[col].mean(), perf[col].std())
    summary = {k: mean_std(k) for k in ['accuracy', 'roc_auc_macro', 'kappa', 'mcc']}

    print("\n" + "="*88 + f"\nCross-validated over {config.N_SPLITS} folds\n" + "="*88)
    for k, (m, s) in summary.items(): print(f"{k.replace('_', ' ').title():<20}: {m:.4f} ± {s:.4f}")

    def merge_pooled(key):
        agg = defaultdict(lambda: [0, 0])
        for m in per_fold:
            for g, (c, t) in m['pooled_groups'][key].items(): agg[g][0] += c; agg[g][1] += t
        return sorted([(g, c/t, t) for g, (c, t) in agg.items() if t>0], key=lambda x: x[1])

    def print_table(title, rows):
        print(f"\n--- {title} ---"); print(f"{'Group':>35s} {'Acc':>8s} {'Count':>8s}")
        for g, acc, cnt in rows:
            if cnt >= config.MIN_SUPPORT: print(f"{g:>35s} {acc:>8.3f} {cnt:>8d}")

    for key, name in [('gender', 'GENDER'), ('age', 'AGE'), ('skin_tone', 'SKIN TONE')]:
        print_table(f"POOLED OVERALL ACCURACY BY {name} (n>={config.MIN_SUPPORT})",
                    merge_pooled(key))

    perf['fold'] = range(1, len(perf) + 1); perf.set_index('fold', inplace=True)
    perf.to_csv(os.path.join(config.RESULTS_DIR, "kfold_performance.csv"))
    pd.DataFrame({k: {'mean': v[0], 'std': v[1]} for k, v in summary.items()}).T.to_csv(
        os.path.join(config.RESULTS_DIR, "kfold_summary.csv"))
    print(f"\nResults, models, and plots saved to '{config.RESULTS_DIR}'")

# -----
# 8) MAIN EXECUTION
# -----
if __name__ == '__main__':

```

```
try:
    df_orig = pd.read_csv(config.ORIGINAL_METADATA_CSV)
    df_std = standardize_metadata_columns(df_orig)
    print(f"Loaded and standardized metadata: {len(df_std)} rows.")
    kfold_cross_val(df_std, config)
except FileNotFoundError:
    print(f"ERROR: Metadata file not found at '{config.ORIGINAL_METADATA_CSV}'")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
    import traceback
    traceback.print_exc()
```

Device: cuda
Model: VGG16 with Concatenation Fusion
K-Fold: N_SPLITS=7 (~Test 14.3%), Val_in_remainder=0.175
Loaded and standardized metadata: 565 rows.

```
#####  
####  
### FOLD 1 ###  
#####  
####  
Unfreezing the last VGG16 conv block for fine-tuning.  
Total params: 164.88M | Trainable params: 31.24M (18.95%)  
Using weighted CrossEntropyLoss for training.
```

```
START TRAINING (VGG16 + Concat) for 200 epochs...  
Epoch 1/200 | Train Loss: 1.9313 Acc: 0.2080 | Val Loss: 1.7965 Acc: 0.3647 | LR: 2.00e-05  
Epoch 2/200 | Train Loss: 1.7555 Acc: 0.3158 | Val Loss: 1.7451 Acc: 0.5176 | LR: 3.00e-05  
Epoch 3/200 | Train Loss: 1.6119 Acc: 0.4612 | Val Loss: 1.6638 Acc: 0.5529 | LR: 4.00e-05  
Epoch 4/200 | Train Loss: 1.4738 Acc: 0.5589 | Val Loss: 1.5812 Acc: 0.6118 | LR: 5.00e-05  
Epoch 5/200 | Train Loss: 1.3502 Acc: 0.6090 | Val Loss: 1.4977 Acc: 0.6706 | LR: 6.00e-05  
Epoch 6/200 | Train Loss: 1.2549 Acc: 0.6441 | Val Loss: 1.4275 Acc: 0.6824 | LR: 7.00e-05  
Epoch 7/200 | Train Loss: 1.1953 Acc: 0.6817 | Val Loss: 1.3525 Acc: 0.7059 | LR: 8.00e-05  
Epoch 8/200 | Train Loss: 1.1745 Acc: 0.7193 | Val Loss: 1.2902 Acc: 0.7294 | LR: 9.00e-05  
Epoch 9/200 | Train Loss: 1.0925 Acc: 0.7719 | Val Loss: 1.2333 Acc: 0.7294 | LR: 1.00e-04  
Epoch 10/200 | Train Loss: 1.0676 Acc: 0.7845 | Val Loss: 1.1745 Acc: 0.7294 | LR: 1.00e-04  
Epoch 11/200 | Train Loss: 1.0342 Acc: 0.8170 | Val Loss: 1.1309 Acc: 0.7529 | LR: 1.00e-04  
Epoch 12/200 | Train Loss: 1.0005 Acc: 0.8296 | Val Loss: 1.1051 Acc: 0.7294 | LR: 1.00e-04  
Epoch 13/200 | Train Loss: 0.9807 Acc: 0.8521 | Val Loss: 1.0796 Acc: 0.7529 | LR: 9.99e-05  
Epoch 14/200 | Train Loss: 0.9572 Acc: 0.8672 | Val Loss: 1.0607 Acc: 0.7647 | LR: 9.99e-05  
Epoch 15/200 | Train Loss: 0.9260 Acc: 0.8672 | Val Loss: 1.0367 Acc: 0.7765 | LR: 9.98e-05  
Epoch 16/200 | Train Loss: 0.9038 Acc: 0.9098 | Val Loss: 1.0219 Acc: 0.7882 | LR: 9.98e-05  
Epoch 17/200 | Train Loss: 0.9100 Acc: 0.9198 | Val Loss: 1.0062 Acc: 0.7882 | LR: 9.97e-05  
Epoch 18/200 | Train Loss: 0.8633 Acc: 0.9123 | Val Loss: 0.9955 Acc: 0.7882 | LR: 9.96e-05  
Epoch 19/200 | Train Loss: 0.8847 Acc: 0.9073 | Val Loss: 0.9913 Acc: 0.7882 | LR: 9.94e-05  
Epoch 20/200 | Train Loss: 0.8944 Acc: 0.9048 | Val Loss: 0.9946 Acc: 0.7882 | LR: 9.93e-05  
Epoch 21/200 | Train Loss: 0.8460 Acc: 0.9298 | Val Loss: 0.9996 Acc: 0.7882 | LR:
```

9.92e-05

Epoch 22/200 | Train Loss: 0.8580 Acc: 0.9173 | Val Loss: 1.0019 Acc: 0.7882 | LR:

9.90e-05

Epoch 23/200 | Train Loss: 0.8315 Acc: 0.9424 | Val Loss: 0.9966 Acc: 0.7765 | LR:

9.88e-05

Epoch 24/200 | Train Loss: 0.8515 Acc: 0.9348 | Val Loss: 0.9899 Acc: 0.8000 | LR:

9.87e-05

Epoch 25/200 | Train Loss: 0.8473 Acc: 0.9424 | Val Loss: 0.9824 Acc: 0.8118 | LR:

9.85e-05

Epoch 26/200 | Train Loss: 0.8365 Acc: 0.9424 | Val Loss: 0.9850 Acc: 0.8118 | LR:

9.83e-05

Epoch 27/200 | Train Loss: 0.8302 Acc: 0.9549 | Val Loss: 0.9764 Acc: 0.8118 | LR:

9.80e-05

Epoch 28/200 | Train Loss: 0.8234 Acc: 0.9398 | Val Loss: 0.9687 Acc: 0.8118 | LR:

9.78e-05

Epoch 29/200 | Train Loss: 0.8318 Acc: 0.9298 | Val Loss: 0.9639 Acc: 0.8000 | LR:

9.76e-05

Epoch 30/200 | Train Loss: 0.8316 Acc: 0.9449 | Val Loss: 0.9590 Acc: 0.8118 | LR:

9.73e-05

Epoch 31/200 | Train Loss: 0.8173 Acc: 0.9549 | Val Loss: 0.9575 Acc: 0.8353 | LR:

9.70e-05

Epoch 32/200 | Train Loss: 0.8332 Acc: 0.9524 | Val Loss: 0.9566 Acc: 0.8118 | LR:

9.67e-05

Epoch 33/200 | Train Loss: 0.7984 Acc: 0.9624 | Val Loss: 0.9544 Acc: 0.8118 | LR:

9.64e-05

Epoch 34/200 | Train Loss: 0.8068 Acc: 0.9674 | Val Loss: 0.9507 Acc: 0.8353 | LR:

9.61e-05

Epoch 35/200 | Train Loss: 0.8088 Acc: 0.9674 | Val Loss: 0.9445 Acc: 0.8471 | LR:

9.58e-05

Epoch 36/200 | Train Loss: 0.8125 Acc: 0.9649 | Val Loss: 0.9424 Acc: 0.8588 | LR:

9.55e-05

Epoch 37/200 | Train Loss: 0.7981 Acc: 0.9649 | Val Loss: 0.9434 Acc: 0.8235 | LR:

9.51e-05

Epoch 38/200 | Train Loss: 0.8019 Acc: 0.9599 | Val Loss: 0.9435 Acc: 0.8000 | LR:

9.47e-05

Epoch 39/200 | Train Loss: 0.8042 Acc: 0.9599 | Val Loss: 0.9417 Acc: 0.8118 | LR:

9.44e-05

Epoch 40/200 | Train Loss: 0.8173 Acc: 0.9649 | Val Loss: 0.9404 Acc: 0.8118 | LR:

9.40e-05

Epoch 41/200 | Train Loss: 0.8084 Acc: 0.9624 | Val Loss: 0.9369 Acc: 0.8000 | LR:

9.36e-05

Epoch 42/200 | Train Loss: 0.7859 Acc: 0.9799 | Val Loss: 0.9369 Acc: 0.8000 | LR:

9.32e-05

Epoch 43/200 | Train Loss: 0.7966 Acc: 0.9724 | Val Loss: 0.9398 Acc: 0.8118 | LR:

9.27e-05

Epoch 44/200 | Train Loss: 0.8184 Acc: 0.9549 | Val Loss: 0.9530 Acc: 0.7882 | LR:

9.23e-05

Epoch 45/200 | Train Loss: 0.8074 Acc: 0.9599 | Val Loss: 0.9597 Acc: 0.7882 | LR:

9.19e-05

Epoch 46/200 | Train Loss: 0.7792 Acc: 0.9825 | Val Loss: 0.9665 Acc: 0.7765 | LR:

9.14e-05

Epoch 47/200 | Train Loss: 0.8035 Acc: 0.9599 | Val Loss: 0.9676 Acc: 0.7882 | LR:

9.09e-05

Epoch 48/200 | Train Loss: 0.7716 Acc: 0.9774 | Val Loss: 0.9621 Acc: 0.8118 | LR:

9.05e-05

Epoch 49/200 | Train Loss: 0.7713 Acc: 0.9699 | Val Loss: 0.9552 Acc: 0.8235 | LR:

9.00e-05

Epoch 50/200 | Train Loss: 0.7740 Acc: 0.9724 | Val Loss: 0.9514 Acc: 0.8235 | LR:

8.95e-05

Epoch 51/200 | Train Loss: 0.7832 Acc: 0.9799 | Val Loss: 0.9523 Acc: 0.8235 | LR:

8.89e-05

Epoch 52/200 | Train Loss: 0.7671 Acc: 0.9850 | Val Loss: 0.9556 Acc: 0.8235 | LR:

8.84e-05

Epoch 53/200 | Train Loss: 0.7777 Acc: 0.9825 | Val Loss: 0.9628 Acc: 0.7882 | LR:

8.79e-05

Epoch 54/200 | Train Loss: 0.7796 Acc: 0.9749 | Val Loss: 0.9648 Acc: 0.7882 | LR:

8.73e-05

Epoch 55/200 | Train Loss: 0.7859 Acc: 0.9724 | Val Loss: 0.9629 Acc: 0.8000 | LR:

8.68e-05

Epoch 56/200 | Train Loss: 0.7737 Acc: 0.9825 | Val Loss: 0.9573 Acc: 0.8118 | LR:

8.62e-05

Epoch 57/200 | Train Loss: 0.7868 Acc: 0.9724 | Val Loss: 0.9464 Acc: 0.8235 | LR:

8.56e-05

Epoch 58/200 | Train Loss: 0.7858 Acc: 0.9749 | Val Loss: 0.9420 Acc: 0.8235 | LR:

8.51e-05

Epoch 59/200 | Train Loss: 0.7620 Acc: 0.9799 | Val Loss: 0.9348 Acc: 0.8353 | LR:

8.45e-05

Epoch 60/200 | Train Loss: 0.7821 Acc: 0.9674 | Val Loss: 0.9342 Acc: 0.8353 | LR:

8.39e-05

Epoch 61/200 | Train Loss: 0.7938 Acc: 0.9674 | Val Loss: 0.9324 Acc: 0.8235 | LR:

8.33e-05

Epoch 62/200 | Train Loss: 0.7714 Acc: 0.9825 | Val Loss: 0.9349 Acc: 0.8235 | LR:

8.26e-05

Epoch 63/200 | Train Loss: 0.7621 Acc: 0.9950 | Val Loss: 0.9350 Acc: 0.8235 | LR:

8.20e-05

Epoch 64/200 | Train Loss: 0.7661 Acc: 0.9850 | Val Loss: 0.9406 Acc: 0.8000 | LR:

8.14e-05

Epoch 65/200 | Train Loss: 0.7657 Acc: 0.9799 | Val Loss: 0.9420 Acc: 0.8000 | LR:

8.07e-05

Epoch 66/200 | Train Loss: 0.7680 Acc: 0.9875 | Val Loss: 0.9443 Acc: 0.8000 | LR:

8.01e-05

Epoch 67/200 | Train Loss: 0.7650 Acc: 0.9825 | Val Loss: 0.9433 Acc: 0.7882 | LR:

7.94e-05

Epoch 68/200 | Train Loss: 0.7696 Acc: 0.9825 | Val Loss: 0.9405 Acc: 0.8000 | LR:

7.87e-05

Epoch 69/200 | Train Loss: 0.7645 Acc: 0.9925 | Val Loss: 0.9398 Acc: 0.8118 | LR:

7.80e-05

Epoch 70/200 | Train Loss: 0.7941 Acc: 0.9674 | Val Loss: 0.9408 Acc: 0.8000 | LR:

7.73e-05

Epoch 71/200 | Train Loss: 0.7549 Acc: 0.9875 | Val Loss: 0.9392 Acc: 0.8000 | LR:

7.67e-05

Epoch 72/200 | Train Loss: 0.7596 Acc: 0.9900 | Val Loss: 0.9355 Acc: 0.8118 | LR:

7.59e-05

Epoch 73/200 | Train Loss: 0.7544 Acc: 0.9925 | Val Loss: 0.9273 Acc: 0.8353 | LR:

7.52e-05

Epoch 74/200 | Train Loss: 0.7637 Acc: 0.9799 | Val Loss: 0.9222 Acc: 0.8235 | LR:

7.45e-05

Epoch 75/200 | Train Loss: 0.7661 Acc: 0.9900 | Val Loss: 0.9208 Acc: 0.8118 | LR:

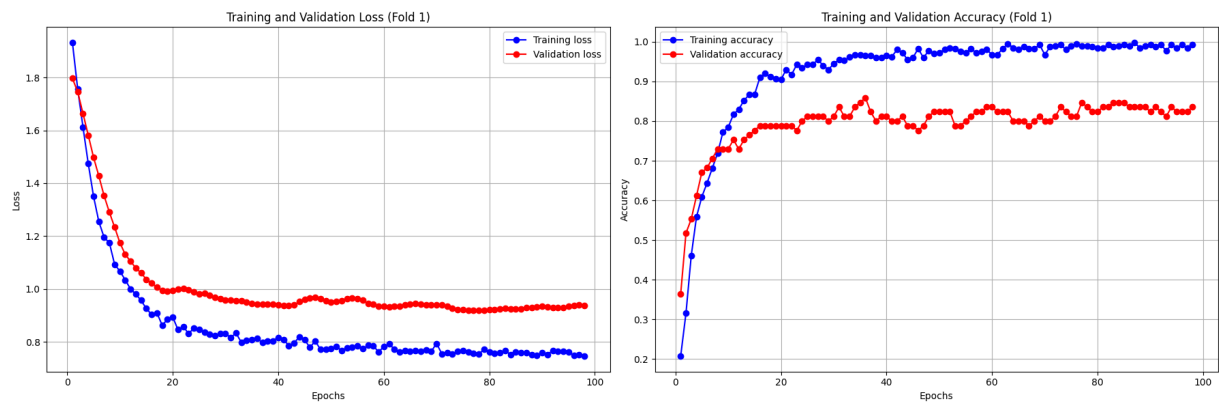
7.38e-05

Epoch 76/200 | Train Loss: 0.7607 Acc: 0.9950 | Val Loss: 0.9204 Acc: 0.8118 | LR:

7.31e-05

Epoch 77/200 | Train Loss: 0.7578 Acc: 0.9900 | Val Loss: 0.9200 Acc: 0.8471 | LR:

7.23e-05
Epoch 78/200 | Train Loss: 0.7536 Acc: 0.9900 | Val Loss: 0.9182 Acc: 0.8353 | LR: 7.16e-05
Epoch 79/200 | Train Loss: 0.7724 Acc: 0.9875 | Val Loss: 0.9191 Acc: 0.8235 | LR: 7.08e-05
Epoch 80/200 | Train Loss: 0.7607 Acc: 0.9850 | Val Loss: 0.9214 Acc: 0.8235 | LR: 7.01e-05
Epoch 81/200 | Train Loss: 0.7569 Acc: 0.9850 | Val Loss: 0.9230 Acc: 0.8353 | LR: 6.93e-05
Epoch 82/200 | Train Loss: 0.7588 Acc: 0.9925 | Val Loss: 0.9233 Acc: 0.8353 | LR: 6.86e-05
Epoch 83/200 | Train Loss: 0.7683 Acc: 0.9875 | Val Loss: 0.9266 Acc: 0.8471 | LR: 6.78e-05
Epoch 84/200 | Train Loss: 0.7518 Acc: 0.9900 | Val Loss: 0.9258 Acc: 0.8471 | LR: 6.70e-05
Epoch 85/200 | Train Loss: 0.7614 Acc: 0.9925 | Val Loss: 0.9248 Acc: 0.8471 | LR: 6.62e-05
Epoch 86/200 | Train Loss: 0.7592 Acc: 0.9900 | Val Loss: 0.9254 Acc: 0.8353 | LR: 6.55e-05
Epoch 87/200 | Train Loss: 0.7584 Acc: 0.9975 | Val Loss: 0.9287 Acc: 0.8353 | LR: 6.47e-05
Epoch 88/200 | Train Loss: 0.7525 Acc: 0.9850 | Val Loss: 0.9298 Acc: 0.8353 | LR: 6.39e-05
Epoch 89/200 | Train Loss: 0.7489 Acc: 0.9900 | Val Loss: 0.9317 Acc: 0.8353 | LR: 6.31e-05
Epoch 90/200 | Train Loss: 0.7589 Acc: 0.9925 | Val Loss: 0.9340 Acc: 0.8235 | LR: 6.23e-05
Epoch 91/200 | Train Loss: 0.7519 Acc: 0.9875 | Val Loss: 0.9323 Acc: 0.8353 | LR: 6.15e-05
Epoch 92/200 | Train Loss: 0.7678 Acc: 0.9925 | Val Loss: 0.9299 Acc: 0.8235 | LR: 6.07e-05
Epoch 93/200 | Train Loss: 0.7634 Acc: 0.9774 | Val Loss: 0.9301 Acc: 0.8118 | LR: 5.99e-05
Epoch 94/200 | Train Loss: 0.7636 Acc: 0.9925 | Val Loss: 0.9300 Acc: 0.8353 | LR: 5.90e-05
Epoch 95/200 | Train Loss: 0.7624 Acc: 0.9850 | Val Loss: 0.9345 Acc: 0.8235 | LR: 5.82e-05
Epoch 96/200 | Train Loss: 0.7500 Acc: 0.9925 | Val Loss: 0.9379 Acc: 0.8235 | LR: 5.74e-05
Epoch 97/200 | Train Loss: 0.7510 Acc: 0.9850 | Val Loss: 0.9392 Acc: 0.8235 | LR: 5.66e-05
Epoch 98/200 | Train Loss: 0.7469 Acc: 0.9925 | Val Loss: 0.9387 Acc: 0.8353 | LR: 5.58e-05
Early stopping triggered.
Training complete in 15.0 minutes. Best Val Loss: 0.9182
Best model for fold saved to: new_results_Multimodal_VGG16_Concat_KFold\best_model_fold_1.pth



Saved training history plot to: new_results_Multimodal_VGG16_Concat_KFold\training_history_fold_1.png

=====

OVERALL PERFORMANCE (FOLD 1)

=====

	precision	recall	f1-score	support
Herpes	0.75	0.86	0.80	7
Melanoma	0.60	1.00	0.75	3
Monkeypox	0.94	0.89	0.91	18
Sarampion	0.86	0.83	0.84	29
Varicela	0.87	0.83	0.85	24
accuracy			0.85	81
macro avg	0.80	0.88	0.83	81
weighted avg	0.86	0.85	0.85	81

--- Overall Scalar Metrics ---

Accuracy: 0.8519

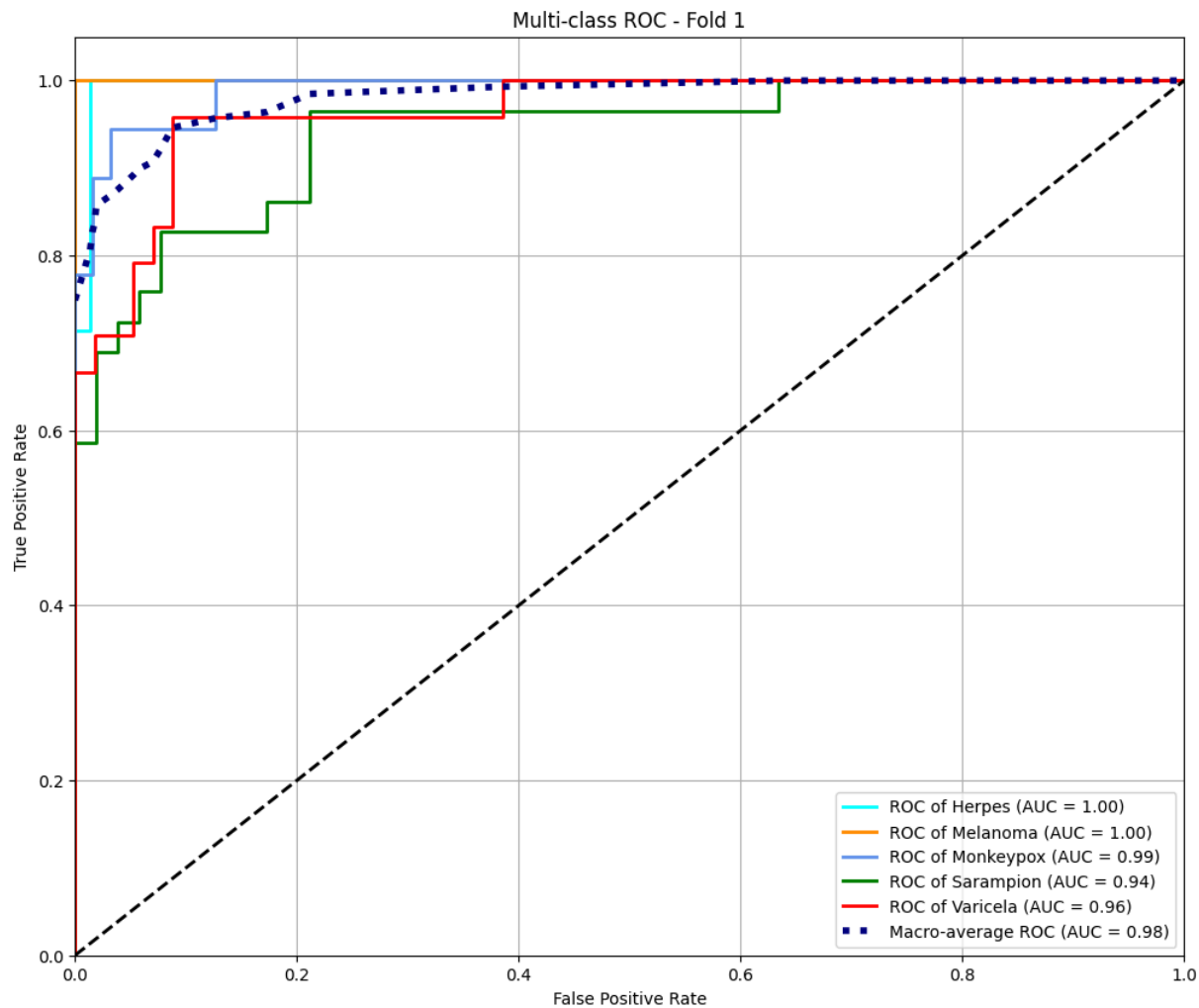
Macro ROC AUC: 0.9780

Cohen's Kappa: 0.7983

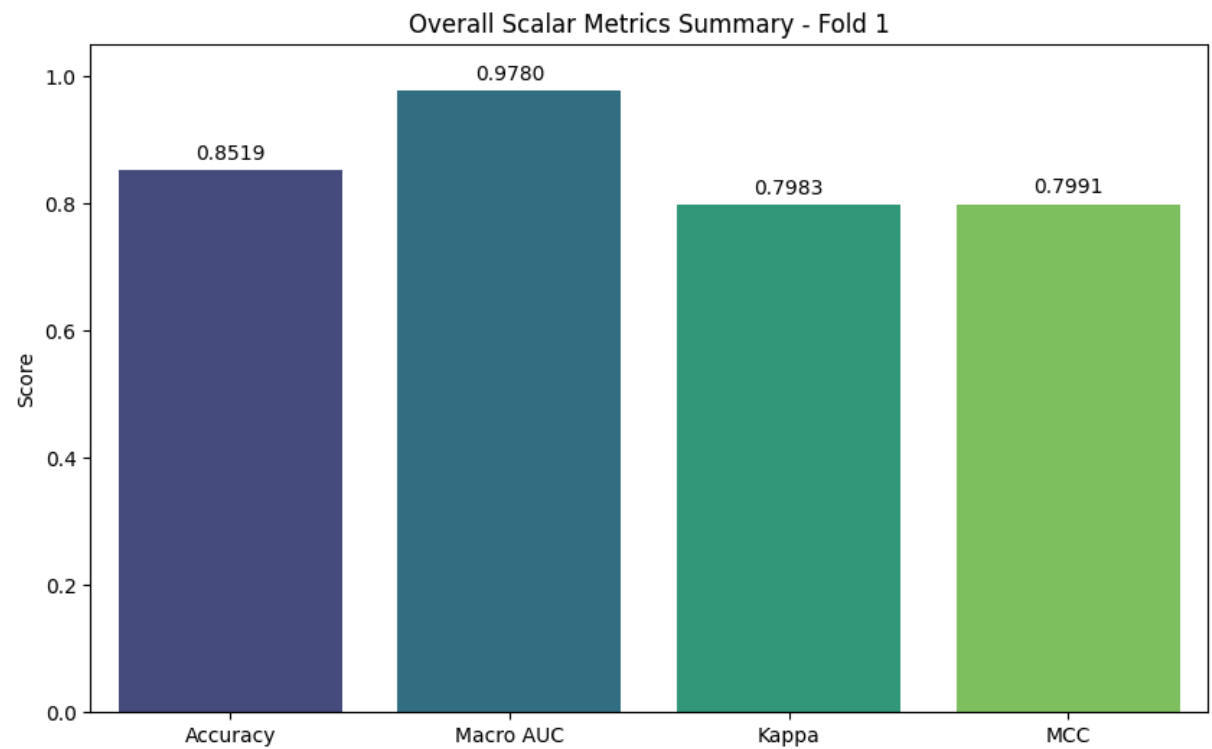
Matthews CorrCoef: 0.7991



Saved normalized confusion matrix to: new_results_Multimodal_VGG16_Concat_KFold\confusion_matrix_fold_1.png



Saved ROC curves plot to: new_results_Multimodal_VGG16_Concat_KFold\roc_curves_fold_1.png



Saved scalar metrics plot to: new_results_Multimodal_VGG16_Concat_KFold\scalar_metrics_fold_1.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	3
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	0.92	0.96	12
Varicela	0.91	1.00	0.95	10
accuracy			0.96	27
macro avg	0.78	0.78	0.78	27
weighted avg	0.97	0.96	0.96	27

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	0.60	0.75	0.67	4
Melanoma	0.60	1.00	0.75	3
Monkeypox	0.93	0.88	0.90	16
Sarampion	0.76	0.76	0.76	17
Varicela	0.83	0.71	0.77	14
accuracy			0.80	54
macro avg	0.75	0.82	0.77	54
weighted avg	0.81	0.80	0.80	54

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.50	1.00	0.67	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.75	0.75	0.75	4
Varicela	1.00	0.75	0.86	4
accuracy			0.82	11
macro avg	0.65	0.70	0.65	11
weighted avg	0.86	0.82	0.83	11

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	0.83	0.83	0.83	6
Melanoma	0.60	1.00	0.75	3
Monkeypox	0.93	0.88	0.90	16
Sarampion	0.88	0.84	0.86	25
Varicela	0.85	0.85	0.85	20
accuracy			0.86	70
macro avg	0.82	0.88	0.84	70
weighted avg	0.87	0.86	0.86	70

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.50	1.00	0.67	1
Varicela	1.00	0.75	0.86	4
accuracy			0.89	9
macro avg	0.70	0.75	0.70	9
weighted avg	0.94	0.89	0.90	9

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.86	0.92	7
Sarampion	0.50	1.00	0.67	1
Varicela	0.00	0.00	0.00	0
accuracy			0.89	9
macro avg	0.50	0.57	0.52	9
weighted avg	0.94	0.89	0.90	9

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	4
Varicela	0.00	0.00	0.00	0
accuracy			1.00	8

macro avg	0.60	0.60	0.60	8
weighted avg	1.00	1.00	1.00	8

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	1.00	1.00	4
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	0.60	0.75	0.67	4
Melanoma	0.60	1.00	0.75	3
Monkeypox	0.67	0.67	0.67	3
Sarampion	0.88	0.74	0.80	19
Varicela	0.85	0.85	0.85	20
accuracy			0.80	49
macro avg	0.72	0.80	0.75	49
weighted avg	0.81	0.80	0.80	49

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.8000
Worst pair: 0-17|female|asian vs 18-59|male|asian
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.6522 EOD: 0.5918
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']
=====
```

```
#####  
####  
### FOLD 2 ###  
#####  
####
```

Unfreezing the last VGG16 conv block for fine-tuning.
Total params: 164.88M | Trainable params: 31.24M (18.95%)
Using weighted CrossEntropyLoss for training.

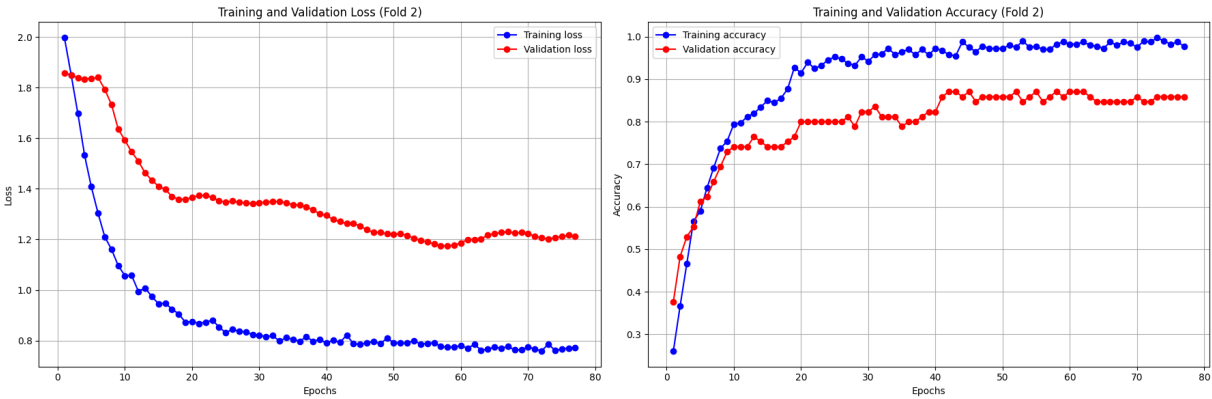
START TRAINING (VGG16 + Concat) for 200 epochs...

Epoch 1/200	Train Loss: 1.9972	Acc: 0.2607	Val Loss: 1.8575	Acc: 0.3765	LR: 2.00e-05
Epoch 2/200	Train Loss: 1.8504	Acc: 0.3659	Val Loss: 1.8504	Acc: 0.4824	LR: 3.00e-05
Epoch 3/200	Train Loss: 1.6973	Acc: 0.4662	Val Loss: 1.8376	Acc: 0.5294	LR: 4.00e-05
Epoch 4/200	Train Loss: 1.5321	Acc: 0.5664	Val Loss: 1.8344	Acc: 0.5529	LR: 5.00e-05
Epoch 5/200	Train Loss: 1.4079	Acc: 0.5890	Val Loss: 1.8359	Acc: 0.6118	LR: 6.00e-05
Epoch 6/200	Train Loss: 1.3034	Acc: 0.6441	Val Loss: 1.8407	Acc: 0.6235	LR: 7.00e-05
Epoch 7/200	Train Loss: 1.2104	Acc: 0.6917	Val Loss: 1.7930	Acc: 0.6588	LR: 8.00e-05
Epoch 8/200	Train Loss: 1.1607	Acc: 0.7368	Val Loss: 1.7333	Acc: 0.6941	LR: 9.00e-05
Epoch 9/200	Train Loss: 1.0967	Acc: 0.7544	Val Loss: 1.6365	Acc: 0.7294	LR: 1.00e-04
Epoch 10/200	Train Loss: 1.0563	Acc: 0.7945	Val Loss: 1.5916	Acc: 0.7412	LR: 1.00e-04
Epoch 11/200	Train Loss: 1.0574	Acc: 0.7970	Val Loss: 1.5456	Acc: 0.7412	LR: 1.00e-04
Epoch 12/200	Train Loss: 0.9946	Acc: 0.8120	Val Loss: 1.5093	Acc: 0.7412	LR: 1.00e-04
Epoch 13/200	Train Loss: 1.0075	Acc: 0.8195	Val Loss: 1.4629	Acc: 0.7647	LR: 9.99e-05
Epoch 14/200	Train Loss: 0.9743	Acc: 0.8346	Val Loss: 1.4341	Acc: 0.7529	LR: 9.99e-05
Epoch 15/200	Train Loss: 0.9453	Acc: 0.8496	Val Loss: 1.4099	Acc: 0.7412	LR: 9.98e-05
Epoch 16/200	Train Loss: 0.9476	Acc: 0.8446	Val Loss: 1.3978	Acc: 0.7412	LR: 9.98e-05
Epoch 17/200	Train Loss: 0.9244	Acc: 0.8546	Val Loss: 1.3696	Acc: 0.7412	LR: 9.97e-05
Epoch 18/200	Train Loss: 0.9054	Acc: 0.8772	Val Loss: 1.3579	Acc: 0.7529	LR: 9.96e-05
Epoch 19/200	Train Loss: 0.8723	Acc: 0.9273	Val Loss: 1.3589	Acc: 0.7647	LR: 9.94e-05
Epoch 20/200	Train Loss: 0.8751	Acc: 0.9148	Val Loss: 1.3654	Acc: 0.8000	LR: 9.93e-05
Epoch 21/200	Train Loss: 0.8676	Acc: 0.9398	Val Loss: 1.3734	Acc: 0.8000	LR: 9.92e-05
Epoch 22/200	Train Loss: 0.8726	Acc: 0.9248	Val Loss: 1.3728	Acc: 0.8000	LR: 9.90e-05
Epoch 23/200	Train Loss: 0.8806	Acc: 0.9323	Val Loss: 1.3667	Acc: 0.8000	LR:

9.88e-05
Epoch 24/200 | Train Loss: 0.8542 Acc: 0.9449 | Val Loss: 1.3523 Acc: 0.8000 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.8301 Acc: 0.9524 | Val Loss: 1.3470 Acc: 0.8000 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.8451 Acc: 0.9474 | Val Loss: 1.3518 Acc: 0.8000 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.8362 Acc: 0.9373 | Val Loss: 1.3459 Acc: 0.8118 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.8336 Acc: 0.9323 | Val Loss: 1.3441 Acc: 0.7882 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.8237 Acc: 0.9524 | Val Loss: 1.3416 Acc: 0.8235 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.8213 Acc: 0.9424 | Val Loss: 1.3457 Acc: 0.8235 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.8151 Acc: 0.9574 | Val Loss: 1.3468 Acc: 0.8353 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.8193 Acc: 0.9599 | Val Loss: 1.3504 Acc: 0.8118 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.7978 Acc: 0.9724 | Val Loss: 1.3486 Acc: 0.8118 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.8110 Acc: 0.9574 | Val Loss: 1.3445 Acc: 0.8118 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.8056 Acc: 0.9649 | Val Loss: 1.3373 Acc: 0.7882 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.7968 Acc: 0.9699 | Val Loss: 1.3360 Acc: 0.8000 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.8156 Acc: 0.9574 | Val Loss: 1.3277 Acc: 0.8000 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.7967 Acc: 0.9699 | Val Loss: 1.3168 Acc: 0.8118 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.8044 Acc: 0.9574 | Val Loss: 1.3001 Acc: 0.8235 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.7895 Acc: 0.9724 | Val Loss: 1.2946 Acc: 0.8235 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.8016 Acc: 0.9674 | Val Loss: 1.2787 Acc: 0.8588 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.7921 Acc: 0.9574 | Val Loss: 1.2702 Acc: 0.8706 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.8217 Acc: 0.9549 | Val Loss: 1.2625 Acc: 0.8706 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.7875 Acc: 0.9875 | Val Loss: 1.2625 Acc: 0.8588 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.7852 Acc: 0.9749 | Val Loss: 1.2527 Acc: 0.8706 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.7913 Acc: 0.9649 | Val Loss: 1.2377 Acc: 0.8471 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.7964 Acc: 0.9774 | Val Loss: 1.2277 Acc: 0.8588 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.7877 Acc: 0.9724 | Val Loss: 1.2283 Acc: 0.8588 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.8094 Acc: 0.9724 | Val Loss: 1.2220 Acc: 0.8588 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.7915 Acc: 0.9724 | Val Loss: 1.2208 Acc: 0.8588 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.7895 Acc: 0.9799 | Val Loss: 1.2223 Acc: 0.8588 | LR:

8.89e-05
Epoch 52/200 | Train Loss: 0.7902 Acc: 0.9749 | Val Loss: 1.2155 Acc: 0.8706 | LR: 8.84e-05
Epoch 53/200 | Train Loss: 0.7998 Acc: 0.9900 | Val Loss: 1.2043 Acc: 0.8471 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.7857 Acc: 0.9749 | Val Loss: 1.1969 Acc: 0.8588 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.7892 Acc: 0.9774 | Val Loss: 1.1901 Acc: 0.8706 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.7912 Acc: 0.9699 | Val Loss: 1.1817 Acc: 0.8471 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.7766 Acc: 0.9699 | Val Loss: 1.1747 Acc: 0.8588 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.7732 Acc: 0.9825 | Val Loss: 1.1752 Acc: 0.8706 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.7735 Acc: 0.9875 | Val Loss: 1.1766 Acc: 0.8588 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.7803 Acc: 0.9825 | Val Loss: 1.1861 Acc: 0.8706 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.7688 Acc: 0.9825 | Val Loss: 1.1996 Acc: 0.8706 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.7852 Acc: 0.9875 | Val Loss: 1.1993 Acc: 0.8706 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.7622 Acc: 0.9799 | Val Loss: 1.2023 Acc: 0.8588 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.7659 Acc: 0.9774 | Val Loss: 1.2162 Acc: 0.8471 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.7740 Acc: 0.9724 | Val Loss: 1.2218 Acc: 0.8471 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.7690 Acc: 0.9875 | Val Loss: 1.2289 Acc: 0.8471 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.7769 Acc: 0.9799 | Val Loss: 1.2296 Acc: 0.8471 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.7627 Acc: 0.9875 | Val Loss: 1.2247 Acc: 0.8471 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.7638 Acc: 0.9850 | Val Loss: 1.2277 Acc: 0.8471 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.7739 Acc: 0.9749 | Val Loss: 1.2222 Acc: 0.8588 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.7669 Acc: 0.9900 | Val Loss: 1.2128 Acc: 0.8471 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.7589 Acc: 0.9875 | Val Loss: 1.2054 Acc: 0.8471 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.7863 Acc: 0.9975 | Val Loss: 1.2015 Acc: 0.8588 | LR: 7.52e-05
Epoch 74/200 | Train Loss: 0.7609 Acc: 0.9900 | Val Loss: 1.2067 Acc: 0.8588 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.7668 Acc: 0.9825 | Val Loss: 1.2119 Acc: 0.8588 | LR: 7.38e-05
Epoch 76/200 | Train Loss: 0.7693 Acc: 0.9875 | Val Loss: 1.2165 Acc: 0.8588 | LR: 7.31e-05
Epoch 77/200 | Train Loss: 0.7711 Acc: 0.9774 | Val Loss: 1.2133 Acc: 0.8588 | LR: 7.23e-05
Early stopping triggered.
Training complete in 12.2 minutes. Best Val Loss: 1.1747

Best model for fold saved to: new_results_Multimodal_VGG16_Concat_KFold\best_model_fold_2.pth



Saved training history plot to: new_results_Multimodal_VGG16_Concat_KFold\training_history_fold_2.png

=====

OVERALL PERFORMANCE (FOLD 2)

=====

	precision	recall	f1-score	support
Herpes	0.58	1.00	0.74	7
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.83	0.91	18
Sarampion	0.89	0.86	0.88	29
Varicela	0.96	0.92	0.94	24
accuracy			0.89	81
macro avg	0.89	0.92	0.89	81
weighted avg	0.91	0.89	0.89	81

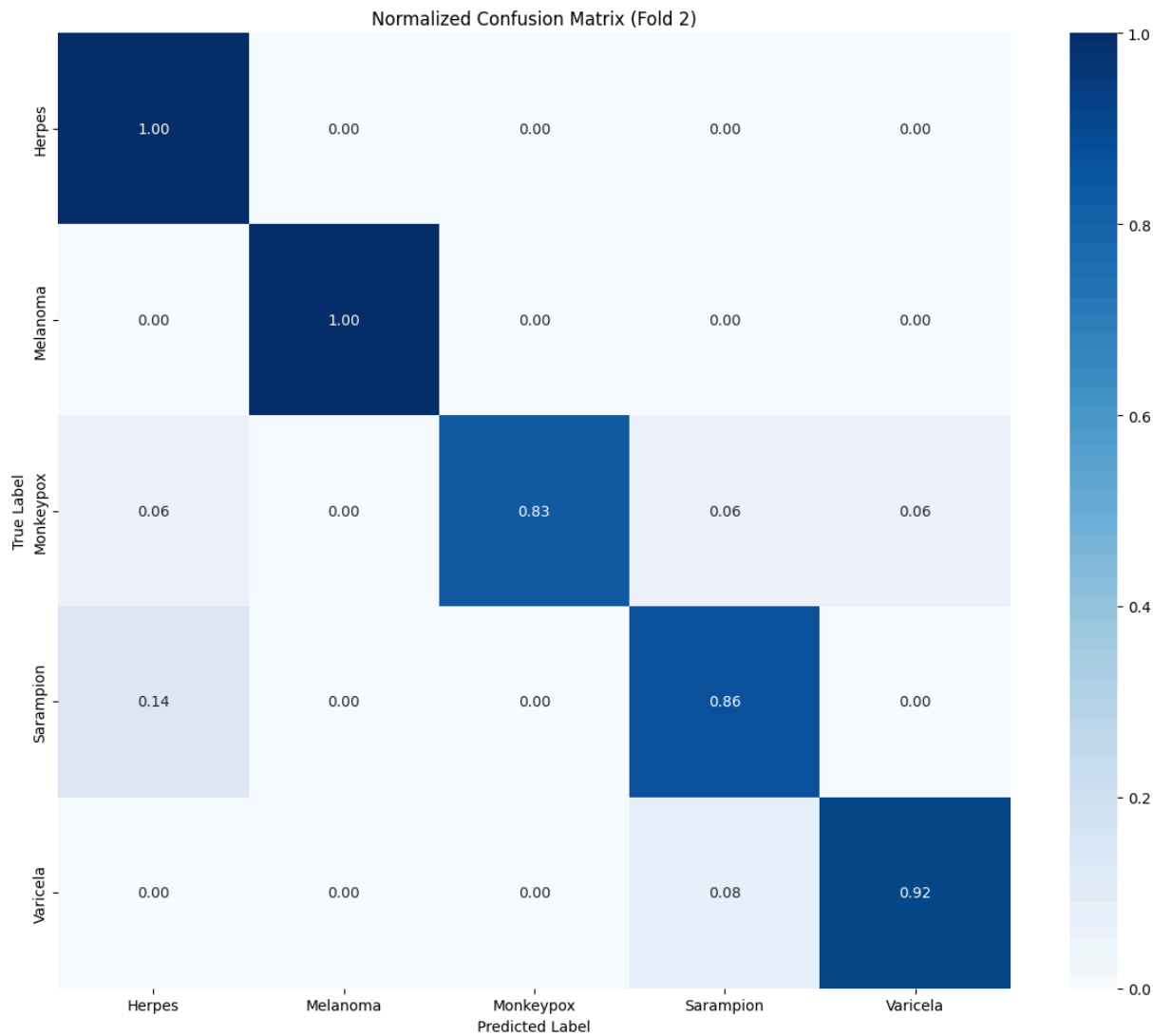
--- Overall Scalar Metrics ---

Accuracy: 0.8889

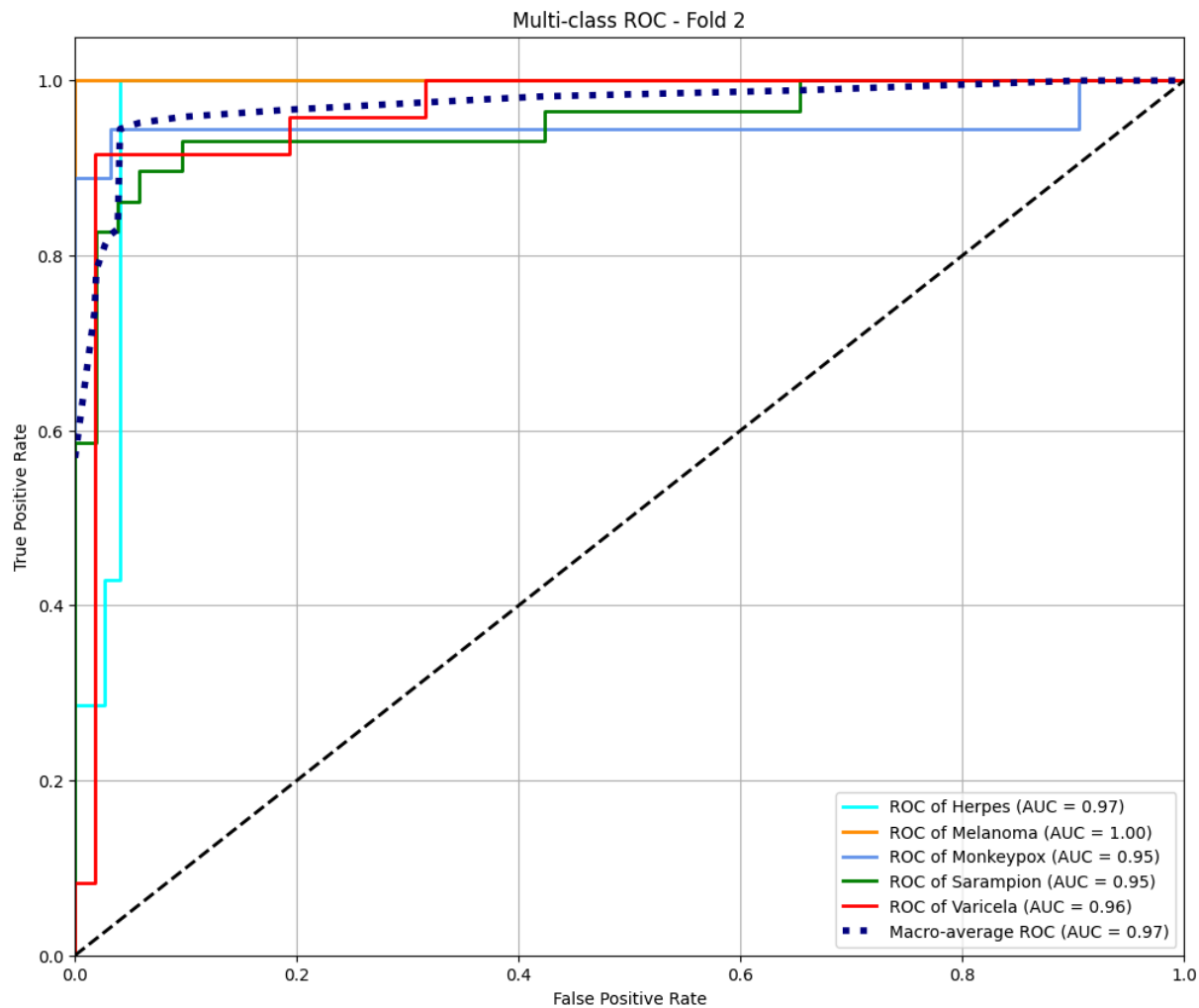
Macro ROC AUC: 0.9673

Cohen's Kappa: 0.8492

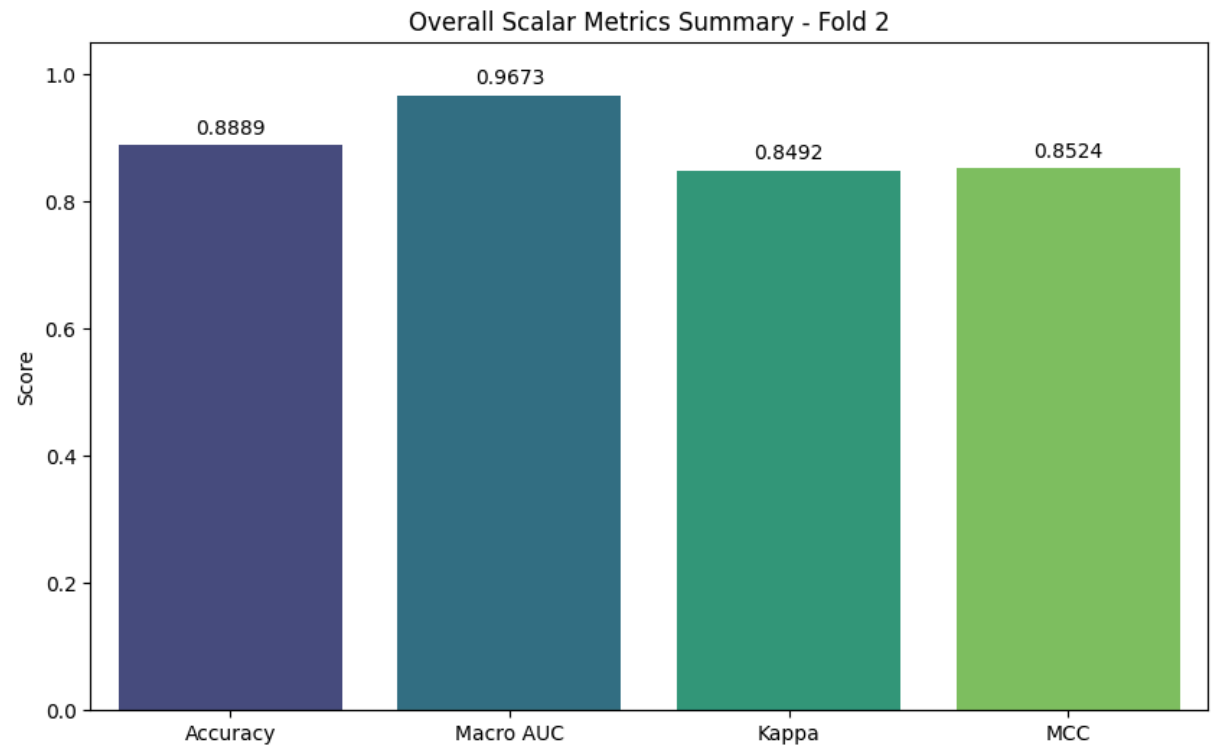
Matthews CorrCoef: 0.8524



Saved normalized confusion matrix to: new_results_Multimodal_VGG16_Concat_KFold\confusion_matrix_fold_2.png



Saved ROC curves plot to: new_results_Multimodal_VGG16_Concat_KFold\roc_curves_fold_2.png



Saved scalar metrics plot to: new_results_Multimodal_VGG16_Concat_KFold\scalar_metrics_fold_2.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.50	0.67	4
Sarampion	0.93	1.00	0.97	14
Varicela	0.93	1.00	0.96	13
accuracy			0.94	33
macro avg	0.77	0.70	0.72	33
weighted avg	0.94	0.94	0.93	33

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	0.50	1.00	0.67	5
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.93	0.96	14
Sarampion	0.85	0.73	0.79	15
Varicela	1.00	0.82	0.90	11
accuracy			0.85	48
macro avg	0.87	0.90	0.86	48
weighted avg	0.90	0.85	0.86	48

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	1
Sarampion	1.00	1.00	1.00	3
Varicela	0.80	1.00	0.89	4
accuracy			0.88	8
macro avg	0.36	0.40	0.38	8
weighted avg	0.78	0.88	0.82	8

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	0.55	1.00	0.71	6
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.88	0.94	17
Sarampion	0.88	0.85	0.86	26
Varicela	1.00	0.90	0.95	20
accuracy			0.89	72
macro avg	0.89	0.93	0.89	72
weighted avg	0.92	0.89	0.90	72

--- Subgroup: 60+ ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	3
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	2
Varicela	1.00	1.00	1.00	2
accuracy			1.00	11
macro avg	0.80	0.80	0.80	11
weighted avg	1.00	1.00	1.00	11

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	2

macro avg	0.20	0.20	0.20	2
weighted avg	1.00	1.00	1.00	2

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	2
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4
macro avg	0.40	0.40	0.40	4
weighted avg	1.00	1.00	1.00	4

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	0.88	1.00	0.93	7
Varicela	0.00	0.00	0.00	0
accuracy			0.91	11
macro avg	0.57	0.53	0.55	11
weighted avg	0.92	0.91	0.90	11

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.00	0.00	0.00	1
Varicela	1.00	1.00	1.00	1
accuracy			0.80	5
macro avg	0.40	0.40	0.40	5
weighted avg	0.80	0.80	0.80	5

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	0.43	1.00	0.60	3
Melanoma	1.00	1.00	1.00	3
Monkeypox	1.00	0.50	0.67	4
Sarampion	0.88	0.82	0.85	17
Varicela	0.95	0.90	0.93	21
accuracy			0.85	48

macro avg	0.85	0.85	0.81	48
weighted avg	0.90	0.85	0.86	48

=====

FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.6667 EOD: 0.8000

Worst pair: 0-17|female|white vs 18-59|male|asian

=====

=====

FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.7500 EOD: 0.6690

Worst pair: 18-59|female|white vs 18-59|male|white

Groups: ['18-59|female|white', '18-59|male|white']

=====

#####

####

FOLD 3

#####

####

Unfreezing the last VGG16 conv block for fine-tuning.

Total params: 164.88M | Trainable params: 31.24M (18.95%)

Using weighted CrossEntropyLoss for training.

START TRAINING (VGG16 + Concat) for 200 epochs...

Epoch 1/200 | Train Loss: 1.9218 Acc: 0.2155 | Val Loss: 1.8093 Acc: 0.4118 | LR: 2.00e-05

Epoch 2/200 | Train Loss: 1.8268 Acc: 0.2682 | Val Loss: 1.7601 Acc: 0.6118 | LR: 3.00e-05

Epoch 3/200 | Train Loss: 1.6386 Acc: 0.4085 | Val Loss: 1.6875 Acc: 0.6471 | LR: 4.00e-05

Epoch 4/200 | Train Loss: 1.5330 Acc: 0.4862 | Val Loss: 1.5803 Acc: 0.6706 | LR: 5.00e-05

Epoch 5/200 | Train Loss: 1.4540 Acc: 0.5213 | Val Loss: 1.4866 Acc: 0.6824 | LR: 6.00e-05

Epoch 6/200 | Train Loss: 1.3471 Acc: 0.5764 | Val Loss: 1.4067 Acc: 0.7176 | LR: 7.00e-05

Epoch 7/200 | Train Loss: 1.2829 Acc: 0.6165 | Val Loss: 1.3509 Acc: 0.7412 | LR: 8.00e-05

Epoch 8/200 | Train Loss: 1.2126 Acc: 0.6541 | Val Loss: 1.2773 Acc: 0.7765 | LR: 9.00e-05

Epoch 9/200 | Train Loss: 1.1340 Acc: 0.7168 | Val Loss: 1.2156 Acc: 0.7412 | LR: 1.00e-04

Epoch 10/200 | Train Loss: 1.0820 Acc: 0.7519 | Val Loss: 1.1638 Acc: 0.7529 | LR: 1.00e-04

Epoch 11/200 | Train Loss: 1.0775 Acc: 0.7794 | Val Loss: 1.1213 Acc: 0.7647 | LR: 1.00e-04

Epoch 12/200 | Train Loss: 0.9952 Acc: 0.7945 | Val Loss: 1.0921 Acc: 0.7647 | LR: 1.00e-04

Epoch 13/200 | Train Loss: 1.0331 Acc: 0.7995 | Val Loss: 1.0730 Acc: 0.7647 | LR: 9.99e-05

Epoch 14/200 | Train Loss: 0.9614 Acc: 0.8496 | Val Loss: 1.0513 Acc: 0.7765 | LR: 9.99e-05

Epoch 15/200 | Train Loss: 0.9470 Acc: 0.8496 | Val Loss: 1.0407 Acc: 0.8000 | LR: 9.98e-05

Epoch 16/200 | Train Loss: 0.9421 Acc: 0.8521 | Val Loss: 1.0378 Acc: 0.8000 | LR:

9.98e-05
Epoch 17/200 | Train Loss: 0.9611 Acc: 0.8797 | Val Loss: 1.0297 Acc: 0.8000 | LR: 9.97e-05
Epoch 18/200 | Train Loss: 0.9211 Acc: 0.8922 | Val Loss: 1.0273 Acc: 0.8118 | LR: 9.96e-05
Epoch 19/200 | Train Loss: 0.9152 Acc: 0.8647 | Val Loss: 1.0232 Acc: 0.8471 | LR: 9.94e-05
Epoch 20/200 | Train Loss: 0.9067 Acc: 0.8747 | Val Loss: 1.0176 Acc: 0.8471 | LR: 9.93e-05
Epoch 21/200 | Train Loss: 0.9223 Acc: 0.8922 | Val Loss: 1.0078 Acc: 0.8588 | LR: 9.92e-05
Epoch 22/200 | Train Loss: 0.8974 Acc: 0.8997 | Val Loss: 0.9897 Acc: 0.8588 | LR: 9.90e-05
Epoch 23/200 | Train Loss: 0.8742 Acc: 0.9023 | Val Loss: 0.9803 Acc: 0.8471 | LR: 9.88e-05
Epoch 24/200 | Train Loss: 0.8656 Acc: 0.9023 | Val Loss: 0.9725 Acc: 0.8706 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.8522 Acc: 0.9173 | Val Loss: 0.9648 Acc: 0.8706 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.8464 Acc: 0.9173 | Val Loss: 0.9620 Acc: 0.8706 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.8591 Acc: 0.9173 | Val Loss: 0.9629 Acc: 0.9059 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.8393 Acc: 0.9248 | Val Loss: 0.9621 Acc: 0.8824 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.8659 Acc: 0.9223 | Val Loss: 0.9592 Acc: 0.8824 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.8553 Acc: 0.9248 | Val Loss: 0.9593 Acc: 0.8941 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.8264 Acc: 0.9348 | Val Loss: 0.9673 Acc: 0.9059 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.8359 Acc: 0.9549 | Val Loss: 0.9701 Acc: 0.9176 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.8406 Acc: 0.9373 | Val Loss: 0.9684 Acc: 0.9176 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.8152 Acc: 0.9549 | Val Loss: 0.9673 Acc: 0.9176 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.8288 Acc: 0.9574 | Val Loss: 0.9647 Acc: 0.9176 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.8151 Acc: 0.9449 | Val Loss: 0.9560 Acc: 0.9294 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.8227 Acc: 0.9348 | Val Loss: 0.9514 Acc: 0.9176 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.8028 Acc: 0.9574 | Val Loss: 0.9625 Acc: 0.9176 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.8253 Acc: 0.9549 | Val Loss: 0.9648 Acc: 0.9059 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.8047 Acc: 0.9599 | Val Loss: 0.9597 Acc: 0.9059 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.8055 Acc: 0.9499 | Val Loss: 0.9560 Acc: 0.9059 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.7962 Acc: 0.9549 | Val Loss: 0.9540 Acc: 0.9059 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.8037 Acc: 0.9574 | Val Loss: 0.9530 Acc: 0.9059 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.8054 Acc: 0.9574 | Val Loss: 0.9521 Acc: 0.9059 | LR:

9.23e-05
Epoch 45/200 | Train Loss: 0.7932 Acc: 0.9749 | Val Loss: 0.9503 Acc: 0.9059 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.7913 Acc: 0.9749 | Val Loss: 0.9506 Acc: 0.9059 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.8062 Acc: 0.9649 | Val Loss: 0.9451 Acc: 0.8941 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.7856 Acc: 0.9799 | Val Loss: 0.9426 Acc: 0.8941 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.7892 Acc: 0.9799 | Val Loss: 0.9340 Acc: 0.8941 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.7912 Acc: 0.9699 | Val Loss: 0.9296 Acc: 0.9059 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.7792 Acc: 0.9699 | Val Loss: 0.9271 Acc: 0.9059 | LR: 8.89e-05
Epoch 52/200 | Train Loss: 0.7855 Acc: 0.9749 | Val Loss: 0.9257 Acc: 0.9059 | LR: 8.84e-05
Epoch 53/200 | Train Loss: 0.7885 Acc: 0.9649 | Val Loss: 0.9246 Acc: 0.8941 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.7793 Acc: 0.9749 | Val Loss: 0.9326 Acc: 0.9176 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.7803 Acc: 0.9699 | Val Loss: 0.9364 Acc: 0.8941 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.7831 Acc: 0.9624 | Val Loss: 0.9413 Acc: 0.9059 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.7791 Acc: 0.9699 | Val Loss: 0.9467 Acc: 0.9294 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.7869 Acc: 0.9774 | Val Loss: 0.9455 Acc: 0.9294 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.7723 Acc: 0.9825 | Val Loss: 0.9523 Acc: 0.9294 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.7835 Acc: 0.9724 | Val Loss: 0.9488 Acc: 0.9176 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.7793 Acc: 0.9724 | Val Loss: 0.9562 Acc: 0.9176 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.7720 Acc: 0.9799 | Val Loss: 0.9575 Acc: 0.8941 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.7781 Acc: 0.9774 | Val Loss: 0.9482 Acc: 0.9176 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.7678 Acc: 0.9799 | Val Loss: 0.9348 Acc: 0.9176 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.7808 Acc: 0.9649 | Val Loss: 0.9183 Acc: 0.9176 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.7750 Acc: 0.9799 | Val Loss: 0.9154 Acc: 0.9176 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.7728 Acc: 0.9799 | Val Loss: 0.9112 Acc: 0.9059 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.7569 Acc: 0.9875 | Val Loss: 0.9113 Acc: 0.8941 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.7656 Acc: 0.9825 | Val Loss: 0.9109 Acc: 0.9059 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.7731 Acc: 0.9799 | Val Loss: 0.9095 Acc: 0.8941 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.7746 Acc: 0.9774 | Val Loss: 0.9078 Acc: 0.8941 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.7720 Acc: 0.9875 | Val Loss: 0.9092 Acc: 0.8941 | LR:

7.59e-05

Epoch 73/200 | Train Loss: 0.7606 Acc: 0.9925 | Val Loss: 0.9124 Acc: 0.8941 | LR:

7.52e-05

Epoch 74/200 | Train Loss: 0.7885 Acc: 0.9825 | Val Loss: 0.9127 Acc: 0.8941 | LR:

7.45e-05

Epoch 75/200 | Train Loss: 0.7674 Acc: 0.9799 | Val Loss: 0.9128 Acc: 0.8941 | LR:

7.38e-05

Epoch 76/200 | Train Loss: 0.7708 Acc: 0.9875 | Val Loss: 0.9173 Acc: 0.8824 | LR:

7.31e-05

Epoch 77/200 | Train Loss: 0.7737 Acc: 0.9825 | Val Loss: 0.9207 Acc: 0.8824 | LR:

7.23e-05

Epoch 78/200 | Train Loss: 0.7615 Acc: 0.9950 | Val Loss: 0.9212 Acc: 0.8824 | LR:

7.16e-05

Epoch 79/200 | Train Loss: 0.7787 Acc: 0.9850 | Val Loss: 0.9196 Acc: 0.8941 | LR:

7.08e-05

Epoch 80/200 | Train Loss: 0.7600 Acc: 0.9799 | Val Loss: 0.9205 Acc: 0.9176 | LR:

7.01e-05

Epoch 81/200 | Train Loss: 0.7644 Acc: 0.9724 | Val Loss: 0.9200 Acc: 0.9176 | LR:

6.93e-05

Epoch 82/200 | Train Loss: 0.7589 Acc: 0.9825 | Val Loss: 0.9221 Acc: 0.9059 | LR:

6.86e-05

Epoch 83/200 | Train Loss: 0.7602 Acc: 0.9799 | Val Loss: 0.9180 Acc: 0.9176 | LR:

6.78e-05

Epoch 84/200 | Train Loss: 0.7594 Acc: 0.9825 | Val Loss: 0.9175 Acc: 0.9176 | LR:

6.70e-05

Epoch 85/200 | Train Loss: 0.7665 Acc: 0.9850 | Val Loss: 0.9184 Acc: 0.9176 | LR:

6.62e-05

Epoch 86/200 | Train Loss: 0.7598 Acc: 0.9774 | Val Loss: 0.9215 Acc: 0.9176 | LR:

6.55e-05

Epoch 87/200 | Train Loss: 0.7535 Acc: 0.9950 | Val Loss: 0.9227 Acc: 0.9176 | LR:

6.47e-05

Epoch 88/200 | Train Loss: 0.7792 Acc: 0.9950 | Val Loss: 0.9183 Acc: 0.9176 | LR:

6.39e-05

Epoch 89/200 | Train Loss: 0.7622 Acc: 0.9875 | Val Loss: 0.9207 Acc: 0.9176 | LR:

6.31e-05

Epoch 90/200 | Train Loss: 0.7509 Acc: 0.9875 | Val Loss: 0.9190 Acc: 0.9059 | LR:

6.23e-05

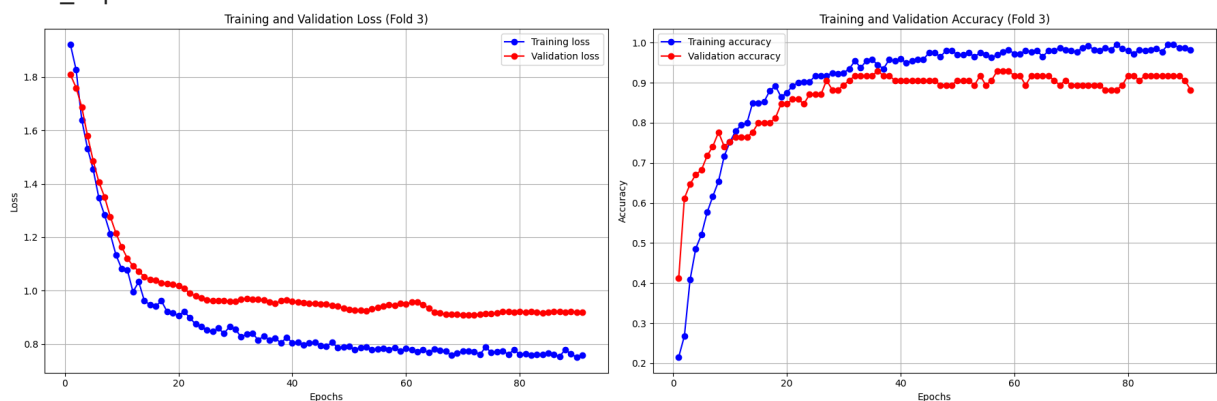
Epoch 91/200 | Train Loss: 0.7574 Acc: 0.9825 | Val Loss: 0.9201 Acc: 0.8824 | LR:

6.15e-05

Early stopping triggered.

Training complete in 11.3 minutes. Best Val Loss: 0.9078

Best model for fold saved to: new_results_Multimodal_VGG16_Concat_KFold\best_model_fold_3.pth



Saved training history plot to: new_results_Multimodal_VGG16_Concat_KFold\training_history_fold_3.png

=====

OVERALL PERFORMANCE (FOLD 3)

=====

	precision	recall	f1-score	support
Herpes	0.57	0.57	0.57	7
Melanoma	0.50	0.33	0.40	3
Monkeypox	0.94	0.94	0.94	18
Sarampion	0.96	0.86	0.91	29
Varicela	0.86	1.00	0.92	24
accuracy			0.88	81
macro avg	0.77	0.74	0.75	81
weighted avg	0.88	0.88	0.87	81

--- Overall Scalar Metrics ---

Accuracy: 0.8765

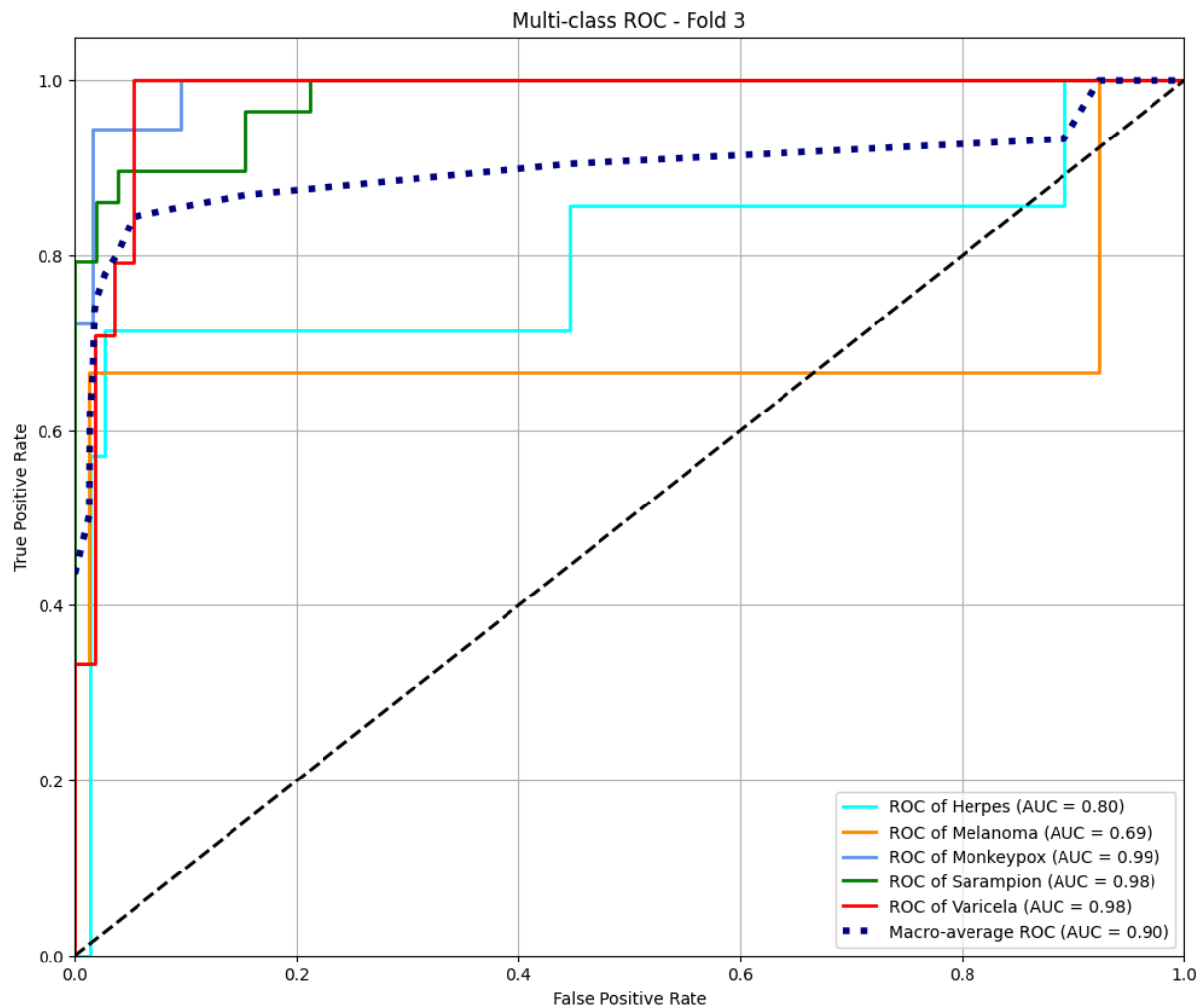
Macro ROC AUC: 0.8871

Cohen's Kappa: 0.8297

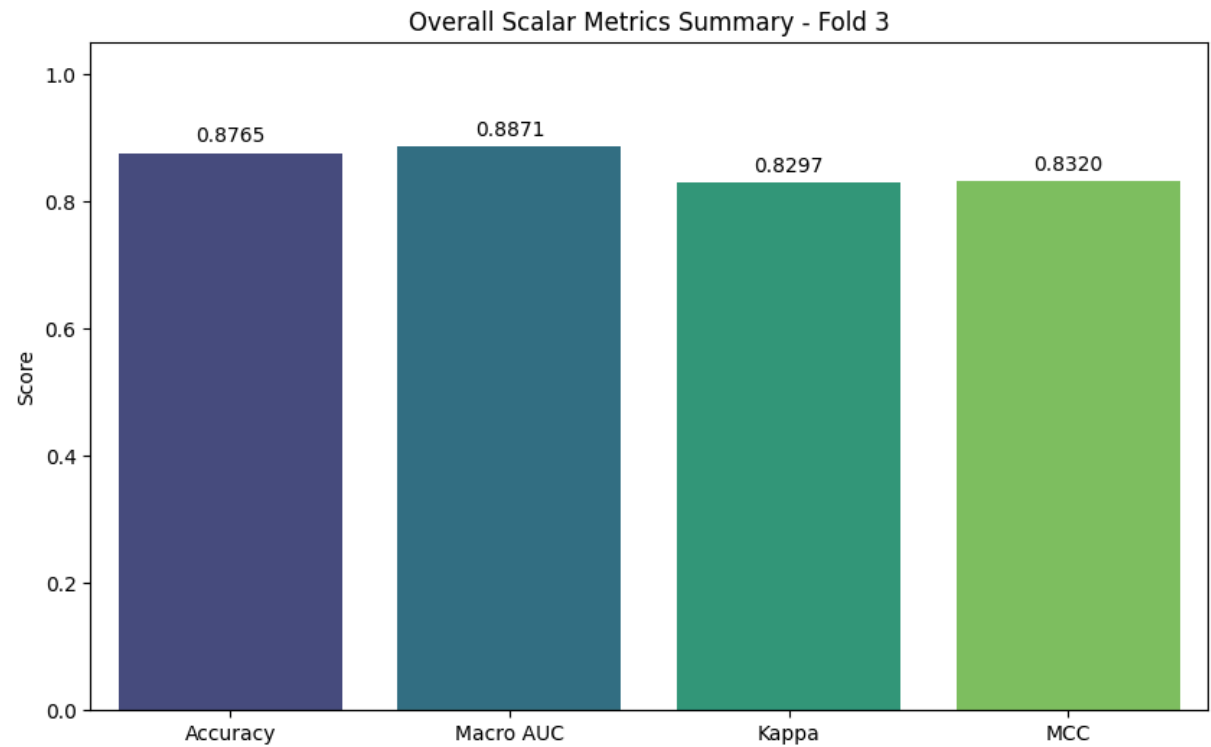
Matthews CorrCoef: 0.8320



Saved normalized confusion matrix to: new_results_Multimodal_VGG16_Concat_KFold\confusion_matrix_fold_3.png



Saved ROC curves plot to: new_results_Multimodal_VGG16_Concat_KFold\roc_curves_fold_3.png



Saved scalar metrics plot to: new_results_Multimodal_VGG16_Concat_KFold\scalar_metrics_fold_3.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	0.50	1.00	0.67	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	0.86	0.93	22
Varicela	0.83	1.00	0.91	10
accuracy			0.92	36
macro avg	0.67	0.77	0.70	36
weighted avg	0.94	0.92	0.92	36

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	0.60	0.50	0.55	6
Melanoma	0.50	0.33	0.40	3
Monkeypox	0.93	0.93	0.93	15
Sarampion	0.86	0.86	0.86	7
Varicela	0.88	1.00	0.93	14
accuracy			0.84	45
macro avg	0.75	0.72	0.73	45
weighted avg	0.83	0.84	0.83	45

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.33	0.50	3
Varicela	0.83	1.00	0.91	5
accuracy			0.75	8
macro avg	0.37	0.27	0.28	8
weighted avg	0.90	0.75	0.76	8

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	0.67	0.57	0.62	7
Melanoma	0.50	0.33	0.40	3
Monkeypox	0.94	0.94	0.94	18
Sarampion	0.96	0.92	0.94	26
Varicela	0.86	1.00	0.93	19
accuracy			0.89	73
macro avg	0.79	0.75	0.77	73
weighted avg	0.88	0.89	0.88	73

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	4
Varicela	0.67	1.00	0.80	2
accuracy			0.82	11
macro avg	0.53	0.60	0.56	11
weighted avg	0.76	0.82	0.78	11

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	3
macro avg	0.20	0.20	0.20	3
weighted avg	1.00	1.00	1.00	3

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.80	1.00	0.89	4
Sarampion	0.00	0.00	0.00	1
Varicela	0.00	0.00	0.00	0
accuracy			0.67	6

macro avg	0.16	0.20	0.18	6
weighted avg	0.53	0.67	0.59	6

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	0.86	0.92	7
Varicela	0.00	0.00	0.00	0
accuracy			0.80	10
macro avg	0.40	0.37	0.38	10
weighted avg	0.90	0.80	0.85	10

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	1.00	1.00	1.00	1
Monkeypox	1.00	0.50	0.67	2
Sarampion	0.50	1.00	0.67	1
Varicela	0.00	0.00	0.00	0
accuracy			0.75	4
macro avg	0.50	0.50	0.47	4
weighted avg	0.88	0.75	0.75	4

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	0.67	1.00	0.80	4
Melanoma	0.00	0.00	0.00	1
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	0.88	0.93	16
Varicela	0.96	1.00	0.98	22
accuracy			0.94	47
macro avg	0.72	0.78	0.74	47
weighted avg	0.93	0.94	0.93	47

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.9000
Worst pair: 18-59|male|middle eastern vs 18-59|male|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.9500 EOD: 0.2154
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']
=====
```

```
#####  
####  
### FOLD 4 ###  
#####  
####
```

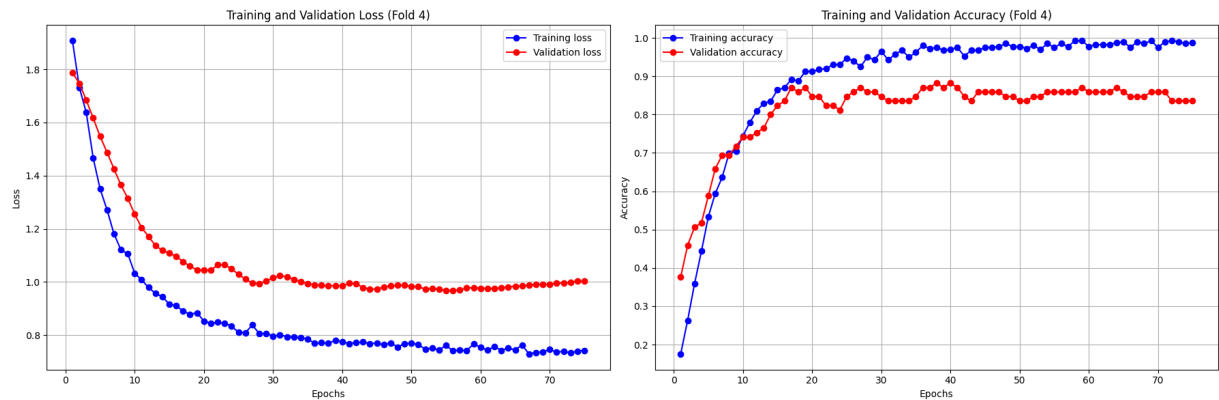
Unfreezing the last VGG16 conv block for fine-tuning.
Total params: 164.88M | Trainable params: 31.24M (18.95%)
Using weighted CrossEntropyLoss for training.

START TRAINING (VGG16 + Concat) for 200 epochs...

Epoch 1/200	Train Loss: 1.9076	Acc: 0.1754	Val Loss: 1.7877	Acc: 0.3765	LR: 2.00e-05
Epoch 2/200	Train Loss: 1.7315	Acc: 0.2632	Val Loss: 1.7472	Acc: 0.4588	LR: 3.00e-05
Epoch 3/200	Train Loss: 1.6375	Acc: 0.3584	Val Loss: 1.6839	Acc: 0.5059	LR: 4.00e-05
Epoch 4/200	Train Loss: 1.4659	Acc: 0.4436	Val Loss: 1.6166	Acc: 0.5176	LR: 5.00e-05
Epoch 5/200	Train Loss: 1.3499	Acc: 0.5338	Val Loss: 1.5477	Acc: 0.5882	LR: 6.00e-05
Epoch 6/200	Train Loss: 1.2714	Acc: 0.5940	Val Loss: 1.4878	Acc: 0.6588	LR: 7.00e-05
Epoch 7/200	Train Loss: 1.1815	Acc: 0.6366	Val Loss: 1.4256	Acc: 0.6941	LR: 8.00e-05
Epoch 8/200	Train Loss: 1.1209	Acc: 0.6992	Val Loss: 1.3652	Acc: 0.6941	LR: 9.00e-05
Epoch 9/200	Train Loss: 1.1057	Acc: 0.7043	Val Loss: 1.3143	Acc: 0.7176	LR: 1.00e-04
Epoch 10/200	Train Loss: 1.0324	Acc: 0.7444	Val Loss: 1.2549	Acc: 0.7412	LR: 1.00e-04
Epoch 11/200	Train Loss: 1.0077	Acc: 0.7794	Val Loss: 1.2039	Acc: 0.7412	LR: 1.00e-04
Epoch 12/200	Train Loss: 0.9792	Acc: 0.8095	Val Loss: 1.1714	Acc: 0.7529	LR: 1.00e-04
Epoch 13/200	Train Loss: 0.9564	Acc: 0.8296	Val Loss: 1.1365	Acc: 0.7647	LR: 9.99e-05
Epoch 14/200	Train Loss: 0.9444	Acc: 0.8346	Val Loss: 1.1184	Acc: 0.8000	LR: 9.99e-05
Epoch 15/200	Train Loss: 0.9161	Acc: 0.8647	Val Loss: 1.1079	Acc: 0.8235	LR: 9.98e-05
Epoch 16/200	Train Loss: 0.9107	Acc: 0.8697	Val Loss: 1.0965	Acc: 0.8353	LR: 9.98e-05
Epoch 17/200	Train Loss: 0.8895	Acc: 0.8922	Val Loss: 1.0761	Acc: 0.8706	LR: 9.97e-05
Epoch 18/200	Train Loss: 0.8788	Acc: 0.8872	Val Loss: 1.0589	Acc: 0.8588	LR: 9.96e-05
Epoch 19/200	Train Loss: 0.8839	Acc: 0.9123	Val Loss: 1.0456	Acc: 0.8706	LR: 9.94e-05
Epoch 20/200	Train Loss: 0.8529	Acc: 0.9123	Val Loss: 1.0444	Acc: 0.8471	LR: 9.93e-05
Epoch 21/200	Train Loss: 0.8443	Acc: 0.9173	Val Loss: 1.0440	Acc: 0.8471	LR: 9.92e-05
Epoch 22/200	Train Loss: 0.8483	Acc: 0.9198	Val Loss: 1.0639	Acc: 0.8235	LR: 9.90e-05
Epoch 23/200	Train Loss: 0.8447	Acc: 0.9298	Val Loss: 1.0653	Acc: 0.8235	LR: 9.90e-05

9.88e-05
Epoch 24/200 | Train Loss: 0.8328 Acc: 0.9298 | Val Loss: 1.0490 Acc: 0.8118 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.8111 Acc: 0.9474 | Val Loss: 1.0286 Acc: 0.8471 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.8088 Acc: 0.9398 | Val Loss: 1.0105 Acc: 0.8588 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.8378 Acc: 0.9248 | Val Loss: 0.9960 Acc: 0.8706 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.8060 Acc: 0.9499 | Val Loss: 0.9936 Acc: 0.8588 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.8065 Acc: 0.9424 | Val Loss: 1.0035 Acc: 0.8588 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.7962 Acc: 0.9649 | Val Loss: 1.0157 Acc: 0.8471 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.8012 Acc: 0.9424 | Val Loss: 1.0228 Acc: 0.8353 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.7938 Acc: 0.9574 | Val Loss: 1.0199 Acc: 0.8353 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.7932 Acc: 0.9674 | Val Loss: 1.0098 Acc: 0.8353 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.7901 Acc: 0.9499 | Val Loss: 1.0011 Acc: 0.8353 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.7855 Acc: 0.9624 | Val Loss: 0.9943 Acc: 0.8471 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.7693 Acc: 0.9799 | Val Loss: 0.9890 Acc: 0.8706 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.7710 Acc: 0.9724 | Val Loss: 0.9878 Acc: 0.8706 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.7695 Acc: 0.9749 | Val Loss: 0.9850 Acc: 0.8824 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.7802 Acc: 0.9674 | Val Loss: 0.9863 Acc: 0.8706 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.7743 Acc: 0.9699 | Val Loss: 0.9855 Acc: 0.8824 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.7679 Acc: 0.9749 | Val Loss: 0.9946 Acc: 0.8706 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.7724 Acc: 0.9524 | Val Loss: 0.9936 Acc: 0.8471 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.7735 Acc: 0.9674 | Val Loss: 0.9788 Acc: 0.8353 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.7681 Acc: 0.9674 | Val Loss: 0.9733 Acc: 0.8588 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.7697 Acc: 0.9749 | Val Loss: 0.9729 Acc: 0.8588 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.7634 Acc: 0.9749 | Val Loss: 0.9808 Acc: 0.8588 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.7705 Acc: 0.9774 | Val Loss: 0.9859 Acc: 0.8588 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.7553 Acc: 0.9850 | Val Loss: 0.9872 Acc: 0.8471 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.7671 Acc: 0.9774 | Val Loss: 0.9869 Acc: 0.8471 | LR: 9.00e-05
Epoch 50/200 | Train Loss: 0.7700 Acc: 0.9774 | Val Loss: 0.9830 Acc: 0.8353 | LR: 8.95e-05
Epoch 51/200 | Train Loss: 0.7644 Acc: 0.9724 | Val Loss: 0.9817 Acc: 0.8353 | LR:

8.89e-05
Epoch 52/200 | Train Loss: 0.7466 Acc: 0.9799 | Val Loss: 0.9728 Acc: 0.8471 | LR: 8.84e-05
Epoch 53/200 | Train Loss: 0.7513 Acc: 0.9699 | Val Loss: 0.9745 Acc: 0.8471 | LR: 8.79e-05
Epoch 54/200 | Train Loss: 0.7436 Acc: 0.9850 | Val Loss: 0.9728 Acc: 0.8588 | LR: 8.73e-05
Epoch 55/200 | Train Loss: 0.7613 Acc: 0.9749 | Val Loss: 0.9676 Acc: 0.8588 | LR: 8.68e-05
Epoch 56/200 | Train Loss: 0.7403 Acc: 0.9850 | Val Loss: 0.9686 Acc: 0.8588 | LR: 8.62e-05
Epoch 57/200 | Train Loss: 0.7444 Acc: 0.9774 | Val Loss: 0.9695 Acc: 0.8588 | LR: 8.56e-05
Epoch 58/200 | Train Loss: 0.7416 Acc: 0.9925 | Val Loss: 0.9768 Acc: 0.8588 | LR: 8.51e-05
Epoch 59/200 | Train Loss: 0.7670 Acc: 0.9925 | Val Loss: 0.9787 Acc: 0.8706 | LR: 8.45e-05
Epoch 60/200 | Train Loss: 0.7536 Acc: 0.9774 | Val Loss: 0.9762 Acc: 0.8588 | LR: 8.39e-05
Epoch 61/200 | Train Loss: 0.7429 Acc: 0.9825 | Val Loss: 0.9759 Acc: 0.8588 | LR: 8.33e-05
Epoch 62/200 | Train Loss: 0.7573 Acc: 0.9825 | Val Loss: 0.9747 Acc: 0.8588 | LR: 8.26e-05
Epoch 63/200 | Train Loss: 0.7424 Acc: 0.9825 | Val Loss: 0.9782 Acc: 0.8588 | LR: 8.20e-05
Epoch 64/200 | Train Loss: 0.7507 Acc: 0.9875 | Val Loss: 0.9811 Acc: 0.8706 | LR: 8.14e-05
Epoch 65/200 | Train Loss: 0.7438 Acc: 0.9900 | Val Loss: 0.9841 Acc: 0.8588 | LR: 8.07e-05
Epoch 66/200 | Train Loss: 0.7629 Acc: 0.9749 | Val Loss: 0.9849 Acc: 0.8471 | LR: 8.01e-05
Epoch 67/200 | Train Loss: 0.7289 Acc: 0.9900 | Val Loss: 0.9869 Acc: 0.8471 | LR: 7.94e-05
Epoch 68/200 | Train Loss: 0.7347 Acc: 0.9850 | Val Loss: 0.9914 Acc: 0.8471 | LR: 7.87e-05
Epoch 69/200 | Train Loss: 0.7367 Acc: 0.9925 | Val Loss: 0.9906 Acc: 0.8588 | LR: 7.80e-05
Epoch 70/200 | Train Loss: 0.7456 Acc: 0.9749 | Val Loss: 0.9914 Acc: 0.8588 | LR: 7.73e-05
Epoch 71/200 | Train Loss: 0.7361 Acc: 0.9900 | Val Loss: 0.9948 Acc: 0.8588 | LR: 7.67e-05
Epoch 72/200 | Train Loss: 0.7383 Acc: 0.9925 | Val Loss: 0.9965 Acc: 0.8353 | LR: 7.59e-05
Epoch 73/200 | Train Loss: 0.7325 Acc: 0.9900 | Val Loss: 0.9985 Acc: 0.8353 | LR: 7.52e-05
Epoch 74/200 | Train Loss: 0.7399 Acc: 0.9850 | Val Loss: 1.0034 Acc: 0.8353 | LR: 7.45e-05
Epoch 75/200 | Train Loss: 0.7425 Acc: 0.9875 | Val Loss: 1.0030 Acc: 0.8353 | LR: 7.38e-05
Early stopping triggered.
Training complete in 10.8 minutes. Best Val Loss: 0.9676
Best model for fold saved to: new_results_Multimodal_VGG16_Concat_KFold\best_model_fold_4.pth



Saved training history plot to: new_results_Multimodal_VGG16_Concat_KFold\training_history_fold_4.png

=====

OVERALL PERFORMANCE (FOLD 4)

=====

	precision	recall	f1-score	support
Herpes	0.50	0.83	0.62	6
Melanoma	0.33	0.50	0.40	2
Monkeypox	1.00	0.79	0.88	19
Sarampion	0.87	0.87	0.87	30
Varicela	0.78	0.75	0.77	24
accuracy			0.80	81
macro avg	0.70	0.75	0.71	81
weighted avg	0.83	0.80	0.81	81

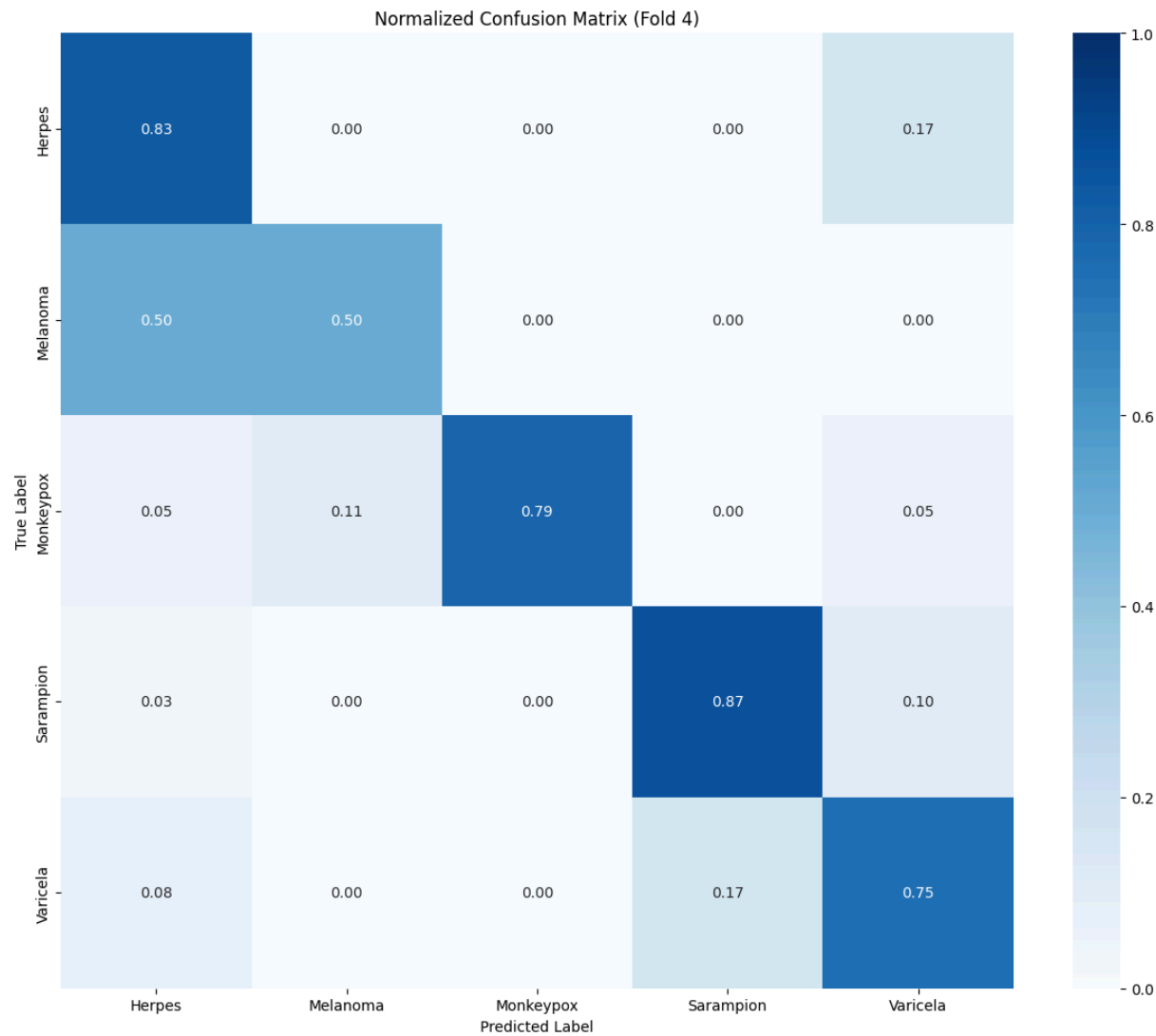
--- Overall Scalar Metrics ---

Accuracy: 0.8025

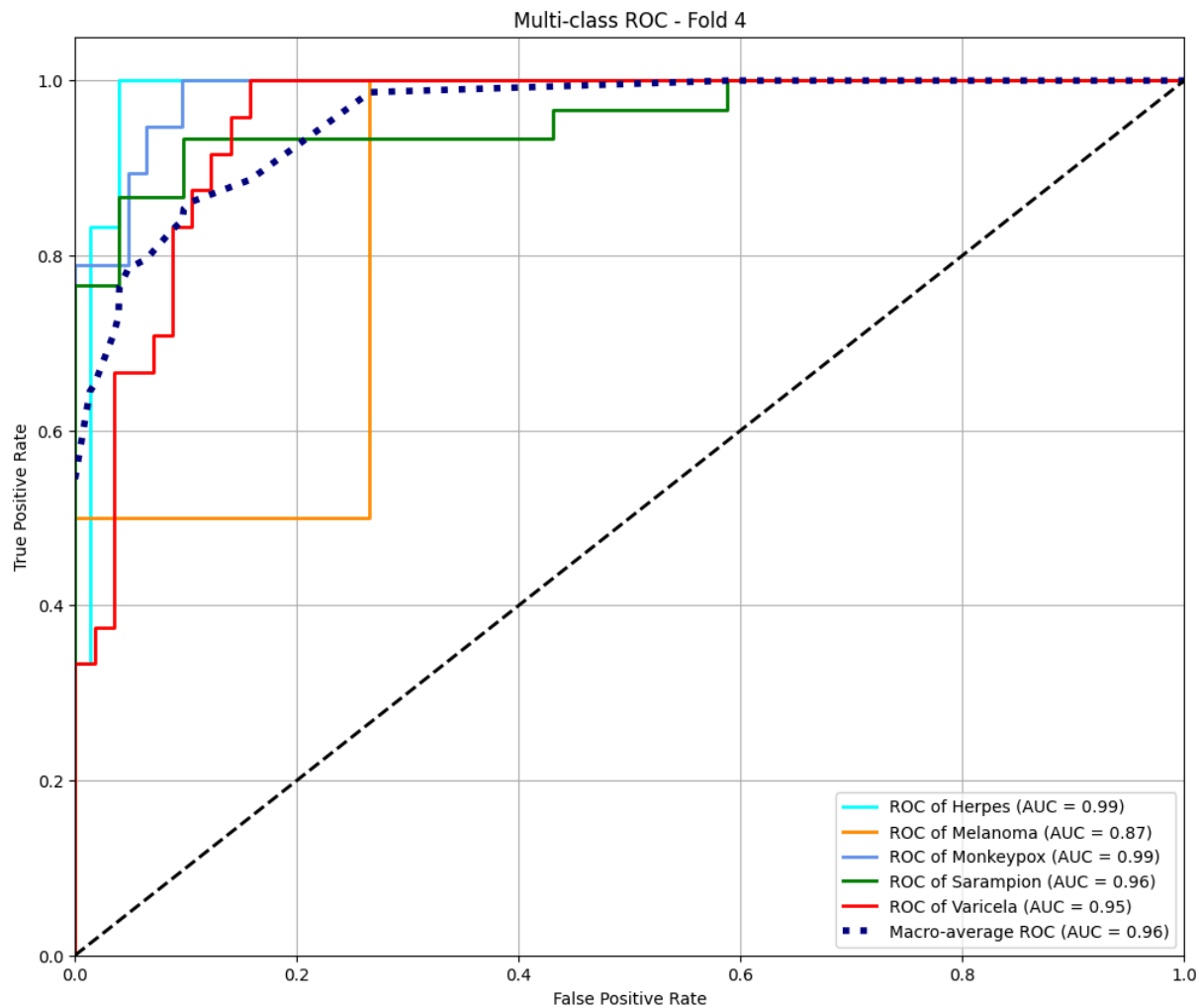
Macro ROC AUC: 0.9498

Cohen's Kappa: 0.7276

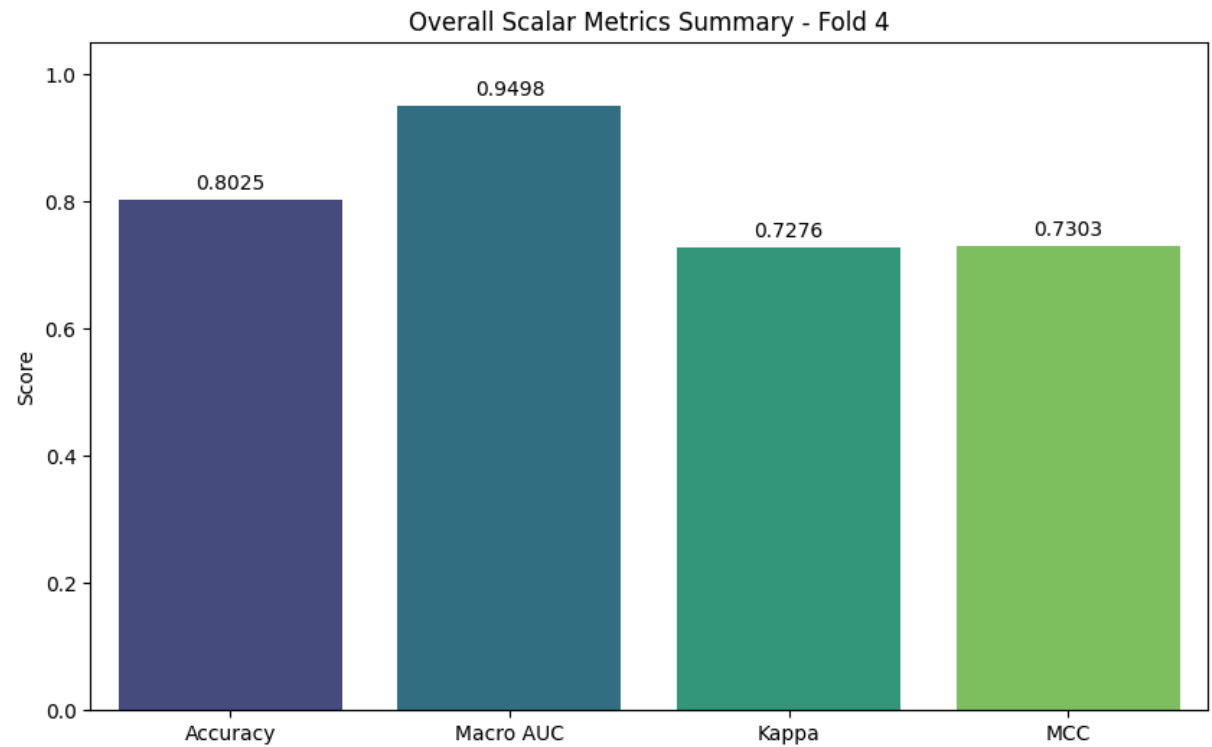
Matthews CorrCoef: 0.7303



Saved normalized confusion matrix to: new_results_Multimodal_VGG16_Concat_KFold\confusion_matrix_fold_4.png



Saved ROC curves plot to: new_results_Multimodal_VGG16_Concat_KFold\roc_curves_fold_4.png



Saved scalar metrics plot to: new_results_Multimodal_VGG16_Concat_KFold\scalar_metrics_fold_4.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	0.50	1.00	0.67	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	0.82	1.00	0.90	14
Varicela	1.00	0.64	0.78	11
accuracy			0.86	29
macro avg	0.66	0.73	0.67	29
weighted avg	0.90	0.86	0.86	29

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	0.50	0.80	0.62	5
Melanoma	0.33	0.50	0.40	2
Monkeypox	1.00	0.75	0.86	16
Sarampion	0.92	0.75	0.83	16
Varicela	0.69	0.85	0.76	13
accuracy			0.77	52
macro avg	0.69	0.73	0.69	52
weighted avg	0.82	0.77	0.78	52

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.50	0.67	2
Varicela	1.00	0.67	0.80	3
accuracy			0.60	5
macro avg	0.40	0.23	0.29	5
weighted avg	1.00	0.60	0.75	5

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	0.62	0.83	0.71	6
Melanoma	0.33	0.50	0.40	2
Monkeypox	1.00	0.79	0.88	19
Sarampion	0.86	0.89	0.88	28
Varicela	0.76	0.76	0.76	21
accuracy			0.82	76
macro avg	0.72	0.76	0.73	76
weighted avg	0.84	0.82	0.82	76

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.67	0.67	0.67	3
Varicela	0.80	0.80	0.80	5
accuracy			0.70	10
macro avg	0.49	0.49	0.49	10
weighted avg	0.70	0.70	0.70	10

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.67	0.80	3
Sarampion	1.00	1.00	1.00	2
Varicela	0.00	0.00	0.00	0
accuracy			0.80	5
macro avg	0.40	0.33	0.36	5
weighted avg	1.00	0.80	0.88	5

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	0.75	0.75	0.75	4
Varicela	0.00	0.00	0.00	1
accuracy			0.71	7

macro avg	0.35	0.35	0.35	7
weighted avg	0.71	0.71	0.71	7

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	4
Varicela	0.00	0.00	0.00	0
accuracy			1.00	7
macro avg	0.40	0.40	0.40	7
weighted avg	1.00	1.00	1.00	7

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	1
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			0.75	4
macro avg	0.40	0.40	0.40	4
weighted avg	0.75	0.75	0.75	4

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	0.62	1.00	0.77	5
Melanoma	0.50	1.00	0.67	1
Monkeypox	1.00	0.62	0.77	8
Sarampion	0.88	0.88	0.88	16
Varicela	0.82	0.78	0.80	18
accuracy			0.81	48
macro avg	0.76	0.86	0.78	48
weighted avg	0.84	0.81	0.81	48

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.7806
Worst pair: 0-17|female|white vs 18-59|male|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.7308 EOD: 0.4361
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']
=====
```

```
#####  
####  
### FOLD 5 ###  
#####  
####
```

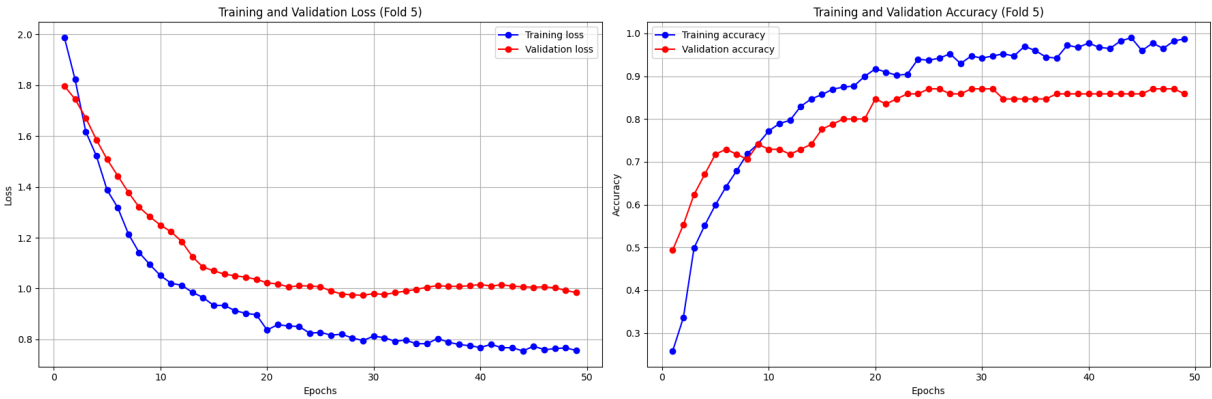
Unfreezing the last VGG16 conv block for fine-tuning.
Total params: 164.88M | Trainable params: 31.24M (18.95%)
Using weighted CrossEntropyLoss for training.

START TRAINING (VGG16 + Concat) for 200 epochs...

Epoch 1/200	Train Loss: 1.9861	Acc: 0.2581	Val Loss: 1.7963	Acc: 0.4941	LR: 2.00e-05
Epoch 2/200	Train Loss: 1.8218	Acc: 0.3358	Val Loss: 1.7445	Acc: 0.5529	LR: 3.00e-05
Epoch 3/200	Train Loss: 1.6169	Acc: 0.4987	Val Loss: 1.6710	Acc: 0.6235	LR: 4.00e-05
Epoch 4/200	Train Loss: 1.5209	Acc: 0.5514	Val Loss: 1.5848	Acc: 0.6706	LR: 5.00e-05
Epoch 5/200	Train Loss: 1.3876	Acc: 0.5990	Val Loss: 1.5080	Acc: 0.7176	LR: 6.00e-05
Epoch 6/200	Train Loss: 1.3180	Acc: 0.6416	Val Loss: 1.4429	Acc: 0.7294	LR: 7.00e-05
Epoch 7/200	Train Loss: 1.2147	Acc: 0.6792	Val Loss: 1.3781	Acc: 0.7176	LR: 8.00e-05
Epoch 8/200	Train Loss: 1.1419	Acc: 0.7193	Val Loss: 1.3202	Acc: 0.7059	LR: 9.00e-05
Epoch 9/200	Train Loss: 1.0947	Acc: 0.7419	Val Loss: 1.2824	Acc: 0.7412	LR: 1.00e-04
Epoch 10/200	Train Loss: 1.0511	Acc: 0.7719	Val Loss: 1.2497	Acc: 0.7294	LR: 1.00e-04
Epoch 11/200	Train Loss: 1.0211	Acc: 0.7895	Val Loss: 1.2232	Acc: 0.7294	LR: 1.00e-04
Epoch 12/200	Train Loss: 1.0128	Acc: 0.7970	Val Loss: 1.1851	Acc: 0.7176	LR: 1.00e-04
Epoch 13/200	Train Loss: 0.9853	Acc: 0.8296	Val Loss: 1.1254	Acc: 0.7294	LR: 9.99e-05
Epoch 14/200	Train Loss: 0.9649	Acc: 0.8471	Val Loss: 1.0841	Acc: 0.7412	LR: 9.99e-05
Epoch 15/200	Train Loss: 0.9337	Acc: 0.8571	Val Loss: 1.0701	Acc: 0.7765	LR: 9.98e-05
Epoch 16/200	Train Loss: 0.9336	Acc: 0.8697	Val Loss: 1.0566	Acc: 0.7882	LR: 9.98e-05
Epoch 17/200	Train Loss: 0.9132	Acc: 0.8747	Val Loss: 1.0504	Acc: 0.8000	LR: 9.97e-05
Epoch 18/200	Train Loss: 0.9027	Acc: 0.8772	Val Loss: 1.0447	Acc: 0.8000	LR: 9.96e-05
Epoch 19/200	Train Loss: 0.8971	Acc: 0.8997	Val Loss: 1.0364	Acc: 0.8000	LR: 9.94e-05
Epoch 20/200	Train Loss: 0.8372	Acc: 0.9173	Val Loss: 1.0225	Acc: 0.8471	LR: 9.93e-05
Epoch 21/200	Train Loss: 0.8581	Acc: 0.9098	Val Loss: 1.0184	Acc: 0.8353	LR: 9.92e-05
Epoch 22/200	Train Loss: 0.8528	Acc: 0.9023	Val Loss: 1.0065	Acc: 0.8471	LR: 9.90e-05
Epoch 23/200	Train Loss: 0.8509	Acc: 0.9048	Val Loss: 1.0109	Acc: 0.8588	LR:

9.88e-05
Epoch 24/200 | Train Loss: 0.8246 Acc: 0.9398 | Val Loss: 1.0102 Acc: 0.8588 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.8279 Acc: 0.9373 | Val Loss: 1.0071 Acc: 0.8706 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.8170 Acc: 0.9424 | Val Loss: 0.9904 Acc: 0.8706 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.8206 Acc: 0.9524 | Val Loss: 0.9784 Acc: 0.8588 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.8055 Acc: 0.9298 | Val Loss: 0.9753 Acc: 0.8588 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.7958 Acc: 0.9474 | Val Loss: 0.9736 Acc: 0.8706 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.8132 Acc: 0.9424 | Val Loss: 0.9802 Acc: 0.8706 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.8065 Acc: 0.9474 | Val Loss: 0.9767 Acc: 0.8706 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.7931 Acc: 0.9524 | Val Loss: 0.9840 Acc: 0.8471 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.7976 Acc: 0.9474 | Val Loss: 0.9901 Acc: 0.8471 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.7831 Acc: 0.9699 | Val Loss: 0.9970 Acc: 0.8471 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.7836 Acc: 0.9599 | Val Loss: 1.0053 Acc: 0.8471 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.8038 Acc: 0.9449 | Val Loss: 1.0116 Acc: 0.8471 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.7892 Acc: 0.9424 | Val Loss: 1.0087 Acc: 0.8588 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.7803 Acc: 0.9724 | Val Loss: 1.0087 Acc: 0.8588 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.7753 Acc: 0.9674 | Val Loss: 1.0110 Acc: 0.8588 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.7684 Acc: 0.9774 | Val Loss: 1.0161 Acc: 0.8588 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.7806 Acc: 0.9674 | Val Loss: 1.0100 Acc: 0.8588 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.7678 Acc: 0.9649 | Val Loss: 1.0153 Acc: 0.8588 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.7675 Acc: 0.9825 | Val Loss: 1.0094 Acc: 0.8588 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.7548 Acc: 0.9900 | Val Loss: 1.0068 Acc: 0.8588 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.7737 Acc: 0.9599 | Val Loss: 1.0053 Acc: 0.8588 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.7597 Acc: 0.9774 | Val Loss: 1.0067 Acc: 0.8706 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.7639 Acc: 0.9649 | Val Loss: 1.0031 Acc: 0.8706 | LR: 9.09e-05
Epoch 48/200 | Train Loss: 0.7669 Acc: 0.9825 | Val Loss: 0.9928 Acc: 0.8706 | LR: 9.05e-05
Epoch 49/200 | Train Loss: 0.7568 Acc: 0.9875 | Val Loss: 0.9841 Acc: 0.8588 | LR: 9.00e-05
Early stopping triggered.
Training complete in 8.2 minutes. Best Val Loss: 0.9736

Best model for fold saved to: new_results_Multimodal_VGG16_Concat_KFold\best_model_fold_5.pth



Saved training history plot to: new_results_Multimodal_VGG16_Concat_KFold\training_history_fold_5.png

=====

OVERALL PERFORMANCE (FOLD 5)

=====

	precision	recall	f1-score	support
Herpes	0.71	0.83	0.77	6
Melanoma	0.67	1.00	0.80	2
Monkeypox	0.89	0.84	0.86	19
Sarampion	0.93	0.83	0.88	30
Varicela	0.88	0.96	0.92	24
accuracy			0.88	81
macro avg	0.82	0.89	0.85	81
weighted avg	0.88	0.88	0.88	81

--- Overall Scalar Metrics ---

Accuracy: 0.8765

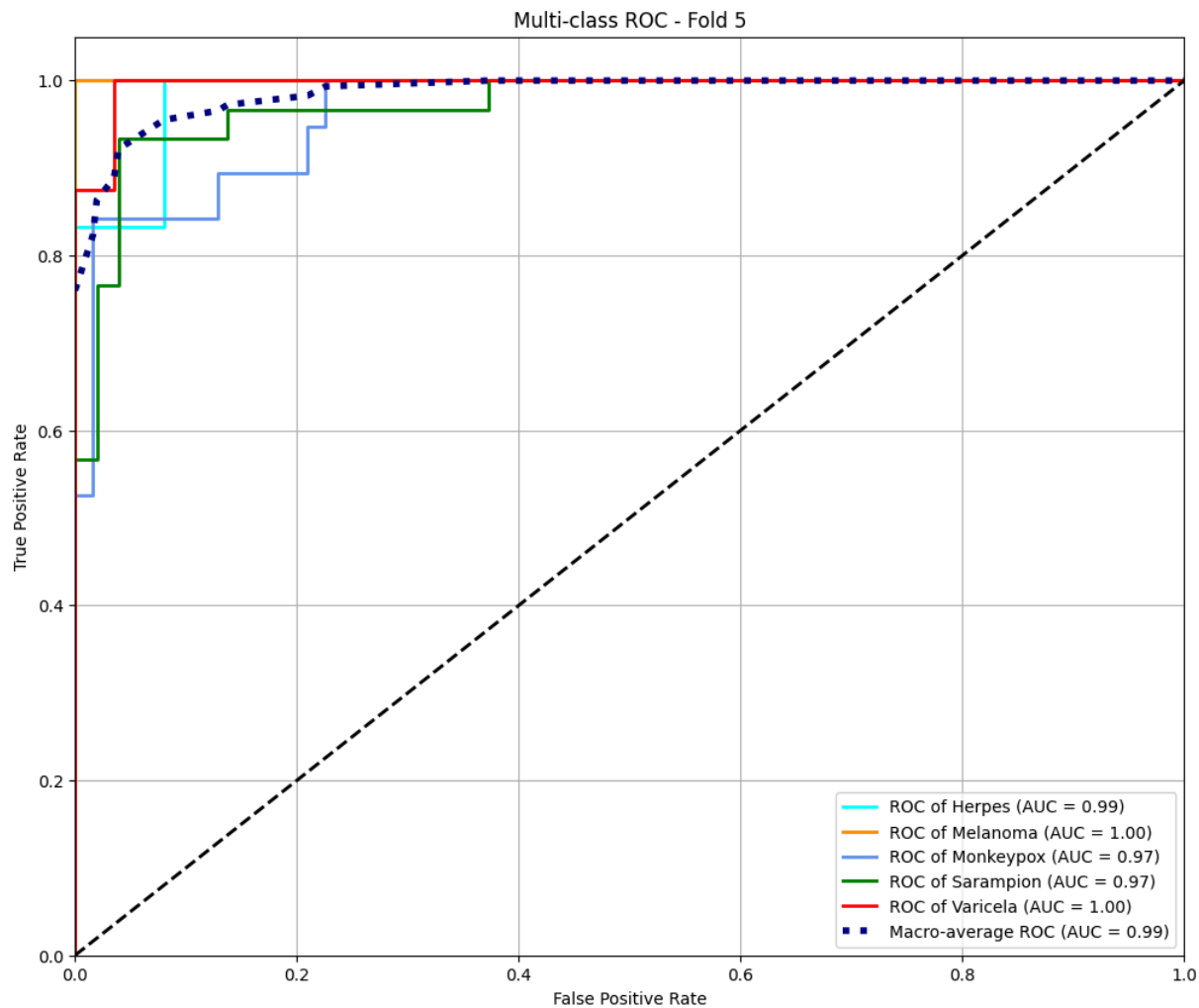
Macro ROC AUC: 0.9840

Cohen's Kappa: 0.8290

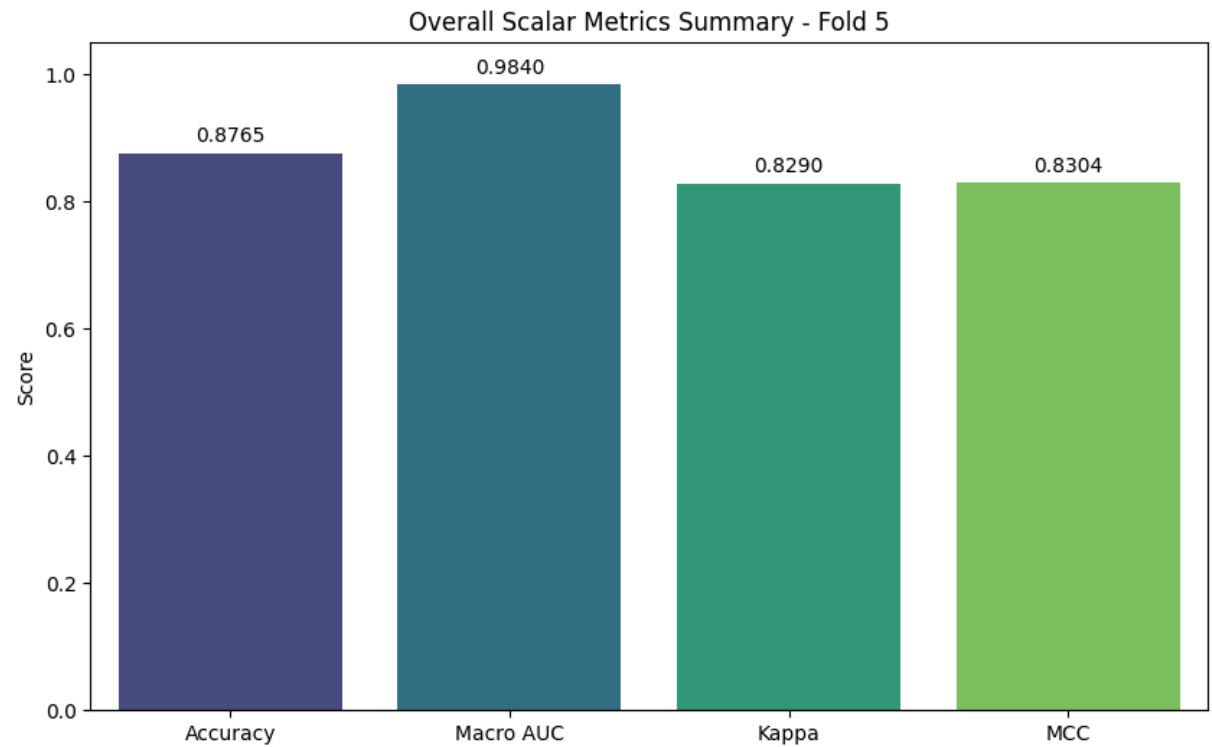
Matthews CorrCoef: 0.8304



Saved normalized confusion matrix to: new_results_Multimodal_VGG16_Concat_KFold\confusion_matrix_fold_5.png



Saved ROC curves plot to: new_results_Multimodal_VGG16_Concat_KFold\roc_curves_fold_5.png



Saved scalar metrics plot to: new_results_Multimodal_VGG16_Concat_KFold\scalar_metrics_fold_5.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	0.50	0.50	0.50	2
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.50	0.67	4
Sarampion	0.88	0.93	0.90	15
Varicela	0.86	0.86	0.86	7
accuracy			0.82	28
macro avg	0.65	0.56	0.59	28
weighted avg	0.86	0.82	0.83	28

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	0.80	1.00	0.89	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.88	0.93	0.90	15
Sarampion	1.00	0.73	0.85	15
Varicela	0.89	1.00	0.94	17
accuracy			0.91	53
macro avg	0.91	0.93	0.92	53
weighted avg	0.92	0.91	0.90	53

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.00	0.00	0.00	1
Varicela	1.00	0.80	0.89	5
accuracy			0.67	6
macro avg	0.20	0.16	0.18	6
weighted avg	0.83	0.67	0.74	6

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	0.83	0.83	0.83	6
Melanoma	0.67	1.00	0.80	2
Monkeypox	0.94	0.84	0.89	19
Sarampion	0.93	0.86	0.89	29
Varicela	0.86	1.00	0.93	19
accuracy			0.89	75
macro avg	0.85	0.91	0.87	75
weighted avg	0.90	0.89	0.89	75

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.50	1.00	0.67	1
Sarampion	0.67	0.50	0.57	4
Varicela	0.75	1.00	0.86	3
accuracy			0.67	9
macro avg	0.38	0.50	0.42	9
weighted avg	0.60	0.67	0.61	9

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.80	1.00	0.89	4
Sarampion	0.00	0.00	0.00	1
Varicela	0.00	0.00	0.00	0
accuracy			0.83	6
macro avg	0.36	0.40	0.38	6
weighted avg	0.70	0.83	0.76	6

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	0.33	0.50	3
Sarampion	0.67	0.50	0.57	4
Varicela	0.75	1.00	0.86	3
accuracy			0.60	10

macro avg	0.48	0.37	0.39	10
weighted avg	0.79	0.60	0.64	10

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	7
Varicela	0.00	0.00	0.00	0
accuracy			1.00	9
macro avg	0.40	0.40	0.40	9
weighted avg	1.00	1.00	1.00	9

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	2
macro avg	0.40	0.40	0.40	2
weighted avg	1.00	1.00	1.00	2

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	0.80	1.00	0.89	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	1.00	0.88	0.93	8
Sarampion	1.00	1.00	1.00	13
Varicela	0.94	0.94	0.94	18
accuracy			0.96	45
macro avg	0.95	0.96	0.95	45
weighted avg	0.96	0.96	0.96	45

=====

FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.9714

Worst pair: 0-17|male|black vs 18-59|male|white

=====

=====

FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.9615 EOD: 0.0000

Worst pair: None vs None

Groups: ['18-59|male|white']

=====

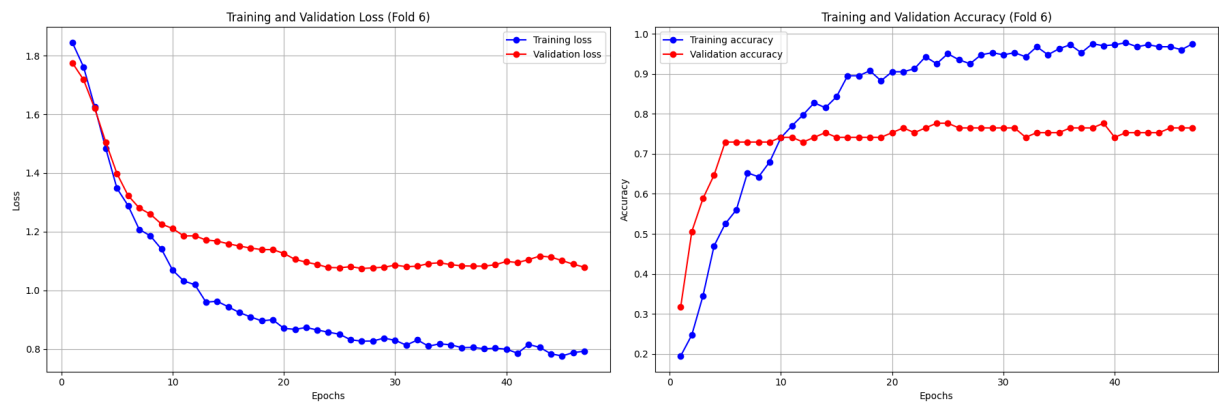
```
#####  
####  
### FOLD 6 ###  
#####  
####
```

Unfreezing the last VGG16 conv block for fine-tuning.
Total params: 164.88M | Trainable params: 31.24M (18.95%)
Using weighted CrossEntropyLoss for training.

START TRAINING (VGG16 + Concat) for 200 epochs...

Epoch 1/200		Train Loss: 1.8442	Acc: 0.1950		Val Loss: 1.7753	Acc: 0.3176		LR: 2.00e-05
Epoch 2/200		Train Loss: 1.7617	Acc: 0.2475		Val Loss: 1.7181	Acc: 0.5059		LR: 3.00e-05
Epoch 3/200		Train Loss: 1.6262	Acc: 0.3450		Val Loss: 1.6201	Acc: 0.5882		LR: 4.00e-05
Epoch 4/200		Train Loss: 1.4831	Acc: 0.4700		Val Loss: 1.5046	Acc: 0.6471		LR: 5.00e-05
Epoch 5/200		Train Loss: 1.3477	Acc: 0.5250		Val Loss: 1.3979	Acc: 0.7294		LR: 6.00e-05
Epoch 6/200		Train Loss: 1.2889	Acc: 0.5600		Val Loss: 1.3231	Acc: 0.7294		LR: 7.00e-05
Epoch 7/200		Train Loss: 1.2076	Acc: 0.6525		Val Loss: 1.2808	Acc: 0.7294		LR: 8.00e-05
Epoch 8/200		Train Loss: 1.1857	Acc: 0.6425		Val Loss: 1.2595	Acc: 0.7294		LR: 9.00e-05
Epoch 9/200		Train Loss: 1.1405	Acc: 0.6800		Val Loss: 1.2259	Acc: 0.7294		LR: 1.00e-04
Epoch 10/200		Train Loss: 1.0687	Acc: 0.7400		Val Loss: 1.2107	Acc: 0.7412		LR: 1.00e-04
Epoch 11/200		Train Loss: 1.0317	Acc: 0.7700		Val Loss: 1.1854	Acc: 0.7412		LR: 1.00e-04
Epoch 12/200		Train Loss: 1.0191	Acc: 0.7975		Val Loss: 1.1858	Acc: 0.7294		LR: 1.00e-04
Epoch 13/200		Train Loss: 0.9596	Acc: 0.8275		Val Loss: 1.1715	Acc: 0.7412		LR: 9.99e-05
Epoch 14/200		Train Loss: 0.9625	Acc: 0.8150		Val Loss: 1.1680	Acc: 0.7529		LR: 9.99e-05
Epoch 15/200		Train Loss: 0.9437	Acc: 0.8425		Val Loss: 1.1586	Acc: 0.7412		LR: 9.98e-05
Epoch 16/200		Train Loss: 0.9240	Acc: 0.8950		Val Loss: 1.1508	Acc: 0.7412		LR: 9.98e-05
Epoch 17/200		Train Loss: 0.9093	Acc: 0.8950		Val Loss: 1.1441	Acc: 0.7412		LR: 9.97e-05
Epoch 18/200		Train Loss: 0.8958	Acc: 0.9075		Val Loss: 1.1388	Acc: 0.7412		LR: 9.96e-05
Epoch 19/200		Train Loss: 0.8996	Acc: 0.8825		Val Loss: 1.1388	Acc: 0.7412		LR: 9.94e-05
Epoch 20/200		Train Loss: 0.8702	Acc: 0.9050		Val Loss: 1.1258	Acc: 0.7529		LR: 9.93e-05
Epoch 21/200		Train Loss: 0.8667	Acc: 0.9050		Val Loss: 1.1053	Acc: 0.7647		LR: 9.92e-05
Epoch 22/200		Train Loss: 0.8732	Acc: 0.9125		Val Loss: 1.0959	Acc: 0.7529		LR: 9.90e-05
Epoch 23/200		Train Loss: 0.8646	Acc: 0.9425		Val Loss: 1.0879	Acc: 0.7647		LR:

9.88e-05
Epoch 24/200 | Train Loss: 0.8570 Acc: 0.9250 | Val Loss: 1.0776 Acc: 0.7765 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.8502 Acc: 0.9500 | Val Loss: 1.0769 Acc: 0.7765 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.8311 Acc: 0.9350 | Val Loss: 1.0805 Acc: 0.7647 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.8265 Acc: 0.9250 | Val Loss: 1.0746 Acc: 0.7647 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.8271 Acc: 0.9475 | Val Loss: 1.0764 Acc: 0.7647 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.8368 Acc: 0.9525 | Val Loss: 1.0787 Acc: 0.7647 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.8294 Acc: 0.9475 | Val Loss: 1.0857 Acc: 0.7647 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.8129 Acc: 0.9525 | Val Loss: 1.0805 Acc: 0.7647 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.8305 Acc: 0.9425 | Val Loss: 1.0823 Acc: 0.7412 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.8094 Acc: 0.9675 | Val Loss: 1.0912 Acc: 0.7529 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.8174 Acc: 0.9475 | Val Loss: 1.0938 Acc: 0.7529 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.8139 Acc: 0.9625 | Val Loss: 1.0875 Acc: 0.7529 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.8043 Acc: 0.9725 | Val Loss: 1.0833 Acc: 0.7647 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.8052 Acc: 0.9525 | Val Loss: 1.0826 Acc: 0.7647 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.8005 Acc: 0.9750 | Val Loss: 1.0823 Acc: 0.7647 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.8024 Acc: 0.9700 | Val Loss: 1.0871 Acc: 0.7765 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.7993 Acc: 0.9725 | Val Loss: 1.0985 Acc: 0.7412 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.7852 Acc: 0.9775 | Val Loss: 1.0946 Acc: 0.7529 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.8151 Acc: 0.9675 | Val Loss: 1.1046 Acc: 0.7529 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.8053 Acc: 0.9725 | Val Loss: 1.1168 Acc: 0.7529 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.7832 Acc: 0.9675 | Val Loss: 1.1132 Acc: 0.7529 | LR: 9.23e-05
Epoch 45/200 | Train Loss: 0.7760 Acc: 0.9675 | Val Loss: 1.1012 Acc: 0.7647 | LR: 9.19e-05
Epoch 46/200 | Train Loss: 0.7876 Acc: 0.9600 | Val Loss: 1.0890 Acc: 0.7647 | LR: 9.14e-05
Epoch 47/200 | Train Loss: 0.7922 Acc: 0.9750 | Val Loss: 1.0787 Acc: 0.7647 | LR: 9.09e-05
Early stopping triggered.
Training complete in 7.8 minutes. Best Val Loss: 1.0746
Best model for fold saved to: new_results_Multimodal_VGG16_Concat_KFold\best_model_fold_6.pth



Saved training history plot to: new_results_Multimodal_VGG16_Concat_KFold\training_history_fold_6.png

=====

OVERALL PERFORMANCE (FOLD 6)

=====

	precision	recall	f1-score	support
Herpes	0.86	1.00	0.92	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.77	0.94	0.85	18
Sarampion	0.91	0.72	0.81	29
Varicela	0.88	0.92	0.90	25
accuracy			0.86	80
macro avg	0.89	0.92	0.90	80
weighted avg	0.87	0.86	0.86	80

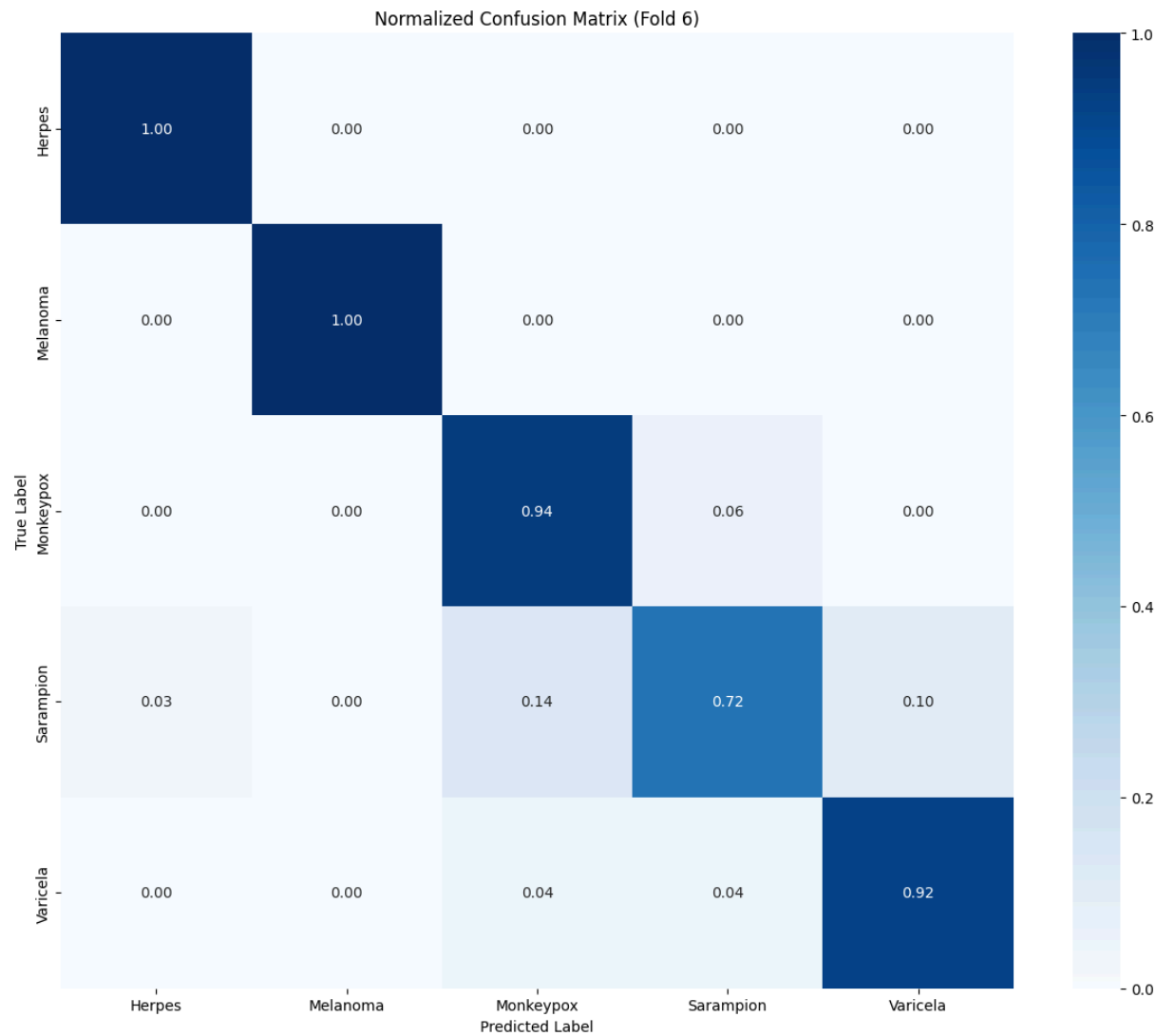
--- Overall Scalar Metrics ---

Accuracy: 0.8625

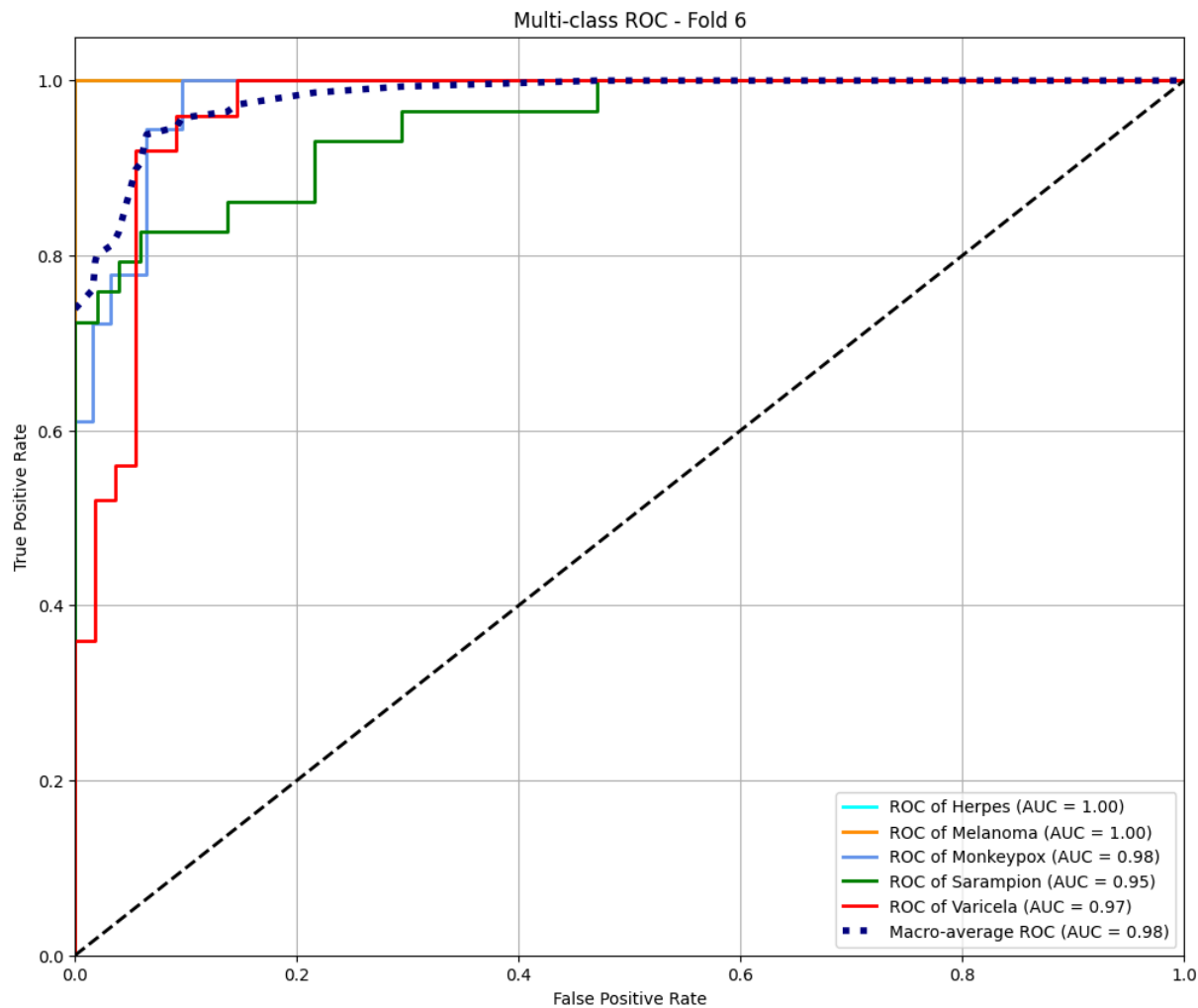
Macro ROC AUC: 0.9794

Cohen's Kappa: 0.8104

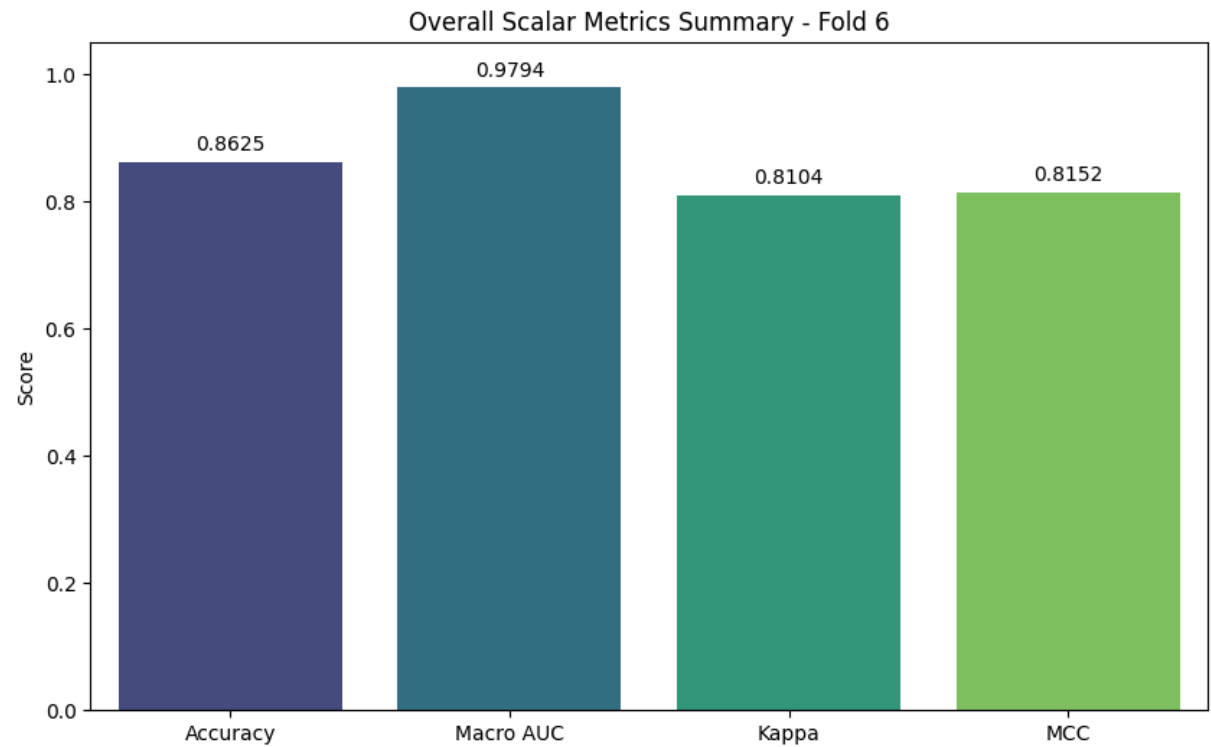
Matthews CorrCoef: 0.8152



Saved normalized confusion matrix to: new_results_Multimodal_VGG16_Concat_KFold\confusion_matrix_fold_6.png



Saved ROC curves plot to: new_results_Multimodal_VGG16_Concat_KFold\roc_curves_fold_6.png



Saved scalar metrics plot to: new_results_Multimodal_VGG16_Concat_KFold\scalar_metrics_fold_6.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.67	1.00	0.80	2
Sarampion	0.91	1.00	0.95	10
Varicela	1.00	0.86	0.92	14
accuracy			0.93	27
macro avg	0.72	0.77	0.74	27
weighted avg	0.94	0.93	0.93	27

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	0.83	1.00	0.91	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.79	0.94	0.86	16
Sarampion	0.92	0.58	0.71	19
Varicela	0.79	1.00	0.88	11
accuracy			0.83	53
macro avg	0.87	0.90	0.87	53
weighted avg	0.85	0.83	0.82	53

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.67	1.00	0.80	2
Sarampion	0.67	0.50	0.57	4
Varicela	0.80	0.80	0.80	5
accuracy			0.75	12
macro avg	0.63	0.66	0.63	12
weighted avg	0.75	0.75	0.74	12

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	0.80	1.00	0.89	4
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.79	0.94	0.86	16
Sarampion	0.95	0.76	0.84	25
Varicela	0.90	0.95	0.93	20
accuracy			0.88	67
macro avg	0.89	0.93	0.90	67
weighted avg	0.89	0.88	0.88	67

--- Subgroup: 60+ ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	1
Varicela	0.83	1.00	0.91	5
accuracy			0.86	7
macro avg	0.37	0.40	0.38	7
weighted avg	0.74	0.86	0.79	7

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	3
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	4

macro avg	0.40	0.40	0.40	4
weighted avg	1.00	1.00	1.00	4

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	0.86	0.92	7
Varicela	1.00	1.00	1.00	1
accuracy			0.90	10
macro avg	0.60	0.57	0.58	10
weighted avg	1.00	0.90	0.95	10

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	1.00	1.00	3
Varicela	0.00	0.00	0.00	0
accuracy			1.00	3
macro avg	0.20	0.20	0.20	3
weighted avg	1.00	1.00	1.00	3

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0.0
Melanoma	0.00	0.00	0.00	0.0
Monkeypox	0.00	0.00	0.00	1.0
Sarampion	0.00	0.00	0.00	0.0
Varicela	0.00	0.00	0.00	0.0
accuracy			0.00	1.0
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.0

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	1.00	1.00	1.00	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.69	1.00	0.81	11
Sarampion	0.92	0.65	0.76	17
Varicela	0.89	0.89	0.89	19
accuracy			0.85	55

macro avg	0.90	0.91	0.89	55
weighted avg	0.88	0.85	0.85	55

=====

FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.9600
Worst pair: 18-59|female|latino hispanic vs 18-59|male|white

=====

=====

FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.8621 EOD: 0.5933
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']

=====

#####

####

FOLD 7

#####

####

Unfreezing the last VGG16 conv block for fine-tuning.
Total params: 164.88M | Trainable params: 31.24M (18.95%)
Using weighted CrossEntropyLoss for training.

START TRAINING (VGG16 + Concat) for 200 epochs...

Epoch 1/200 | Train Loss: 1.9775 Acc: 0.2125 | Val Loss: 1.7974 Acc: 0.3176 | LR: 2.00e-05

Epoch 2/200 | Train Loss: 1.8881 Acc: 0.2600 | Val Loss: 1.7666 Acc: 0.5412 | LR: 3.00e-05

Epoch 3/200 | Train Loss: 1.6852 Acc: 0.4350 | Val Loss: 1.7176 Acc: 0.6000 | LR: 4.00e-05

Epoch 4/200 | Train Loss: 1.6186 Acc: 0.4925 | Val Loss: 1.6472 Acc: 0.6706 | LR: 5.00e-05

Epoch 5/200 | Train Loss: 1.4116 Acc: 0.5725 | Val Loss: 1.5854 Acc: 0.6824 | LR: 6.00e-05

Epoch 6/200 | Train Loss: 1.3105 Acc: 0.6200 | Val Loss: 1.5398 Acc: 0.7059 | LR: 7.00e-05

Epoch 7/200 | Train Loss: 1.2448 Acc: 0.6725 | Val Loss: 1.4823 Acc: 0.7059 | LR: 8.00e-05

Epoch 8/200 | Train Loss: 1.1931 Acc: 0.6975 | Val Loss: 1.4466 Acc: 0.7176 | LR: 9.00e-05

Epoch 9/200 | Train Loss: 1.0904 Acc: 0.7350 | Val Loss: 1.4026 Acc: 0.7176 | LR: 1.00e-04

Epoch 10/200 | Train Loss: 1.0475 Acc: 0.7900 | Val Loss: 1.3403 Acc: 0.7294 | LR: 1.00e-04

Epoch 11/200 | Train Loss: 1.0280 Acc: 0.8075 | Val Loss: 1.2759 Acc: 0.7294 | LR: 1.00e-04

Epoch 12/200 | Train Loss: 1.0143 Acc: 0.8175 | Val Loss: 1.2123 Acc: 0.7529 | LR: 1.00e-04

Epoch 13/200 | Train Loss: 0.9861 Acc: 0.8275 | Val Loss: 1.1612 Acc: 0.7647 | LR: 9.99e-05

Epoch 14/200 | Train Loss: 0.9545 Acc: 0.8700 | Val Loss: 1.1352 Acc: 0.7765 | LR: 9.99e-05

Epoch 15/200 | Train Loss: 0.9338 Acc: 0.8625 | Val Loss: 1.1036 Acc: 0.7765 | LR: 9.98e-05

Epoch 16/200 | Train Loss: 0.9356 Acc: 0.8800 | Val Loss: 1.0824 Acc: 0.7765 | LR:

9.98e-05
Epoch 17/200 | Train Loss: 0.8979 Acc: 0.9000 | Val Loss: 1.0770 Acc: 0.8000 | LR: 9.97e-05
Epoch 18/200 | Train Loss: 0.8810 Acc: 0.9075 | Val Loss: 1.0662 Acc: 0.8000 | LR: 9.96e-05
Epoch 19/200 | Train Loss: 0.8853 Acc: 0.9175 | Val Loss: 1.0685 Acc: 0.7882 | LR: 9.94e-05
Epoch 20/200 | Train Loss: 0.8712 Acc: 0.9175 | Val Loss: 1.0565 Acc: 0.7647 | LR: 9.93e-05
Epoch 21/200 | Train Loss: 0.8708 Acc: 0.9200 | Val Loss: 1.0618 Acc: 0.7647 | LR: 9.92e-05
Epoch 22/200 | Train Loss: 0.8639 Acc: 0.9275 | Val Loss: 1.0688 Acc: 0.7647 | LR: 9.90e-05
Epoch 23/200 | Train Loss: 0.8736 Acc: 0.9225 | Val Loss: 1.0591 Acc: 0.7765 | LR: 9.88e-05
Epoch 24/200 | Train Loss: 0.8520 Acc: 0.9300 | Val Loss: 1.0587 Acc: 0.7882 | LR: 9.87e-05
Epoch 25/200 | Train Loss: 0.8472 Acc: 0.9175 | Val Loss: 1.0473 Acc: 0.8118 | LR: 9.85e-05
Epoch 26/200 | Train Loss: 0.8414 Acc: 0.9400 | Val Loss: 1.0476 Acc: 0.8118 | LR: 9.83e-05
Epoch 27/200 | Train Loss: 0.8386 Acc: 0.9375 | Val Loss: 1.0383 Acc: 0.8353 | LR: 9.80e-05
Epoch 28/200 | Train Loss: 0.8223 Acc: 0.9575 | Val Loss: 1.0247 Acc: 0.8235 | LR: 9.78e-05
Epoch 29/200 | Train Loss: 0.8253 Acc: 0.9300 | Val Loss: 1.0149 Acc: 0.8353 | LR: 9.76e-05
Epoch 30/200 | Train Loss: 0.8137 Acc: 0.9450 | Val Loss: 1.0214 Acc: 0.8235 | LR: 9.73e-05
Epoch 31/200 | Train Loss: 0.8368 Acc: 0.9475 | Val Loss: 1.0171 Acc: 0.8235 | LR: 9.70e-05
Epoch 32/200 | Train Loss: 0.8364 Acc: 0.9450 | Val Loss: 1.0169 Acc: 0.8235 | LR: 9.67e-05
Epoch 33/200 | Train Loss: 0.8223 Acc: 0.9575 | Val Loss: 1.0193 Acc: 0.8235 | LR: 9.64e-05
Epoch 34/200 | Train Loss: 0.8109 Acc: 0.9500 | Val Loss: 1.0135 Acc: 0.8353 | LR: 9.61e-05
Epoch 35/200 | Train Loss: 0.8083 Acc: 0.9525 | Val Loss: 1.0279 Acc: 0.8353 | LR: 9.58e-05
Epoch 36/200 | Train Loss: 0.7906 Acc: 0.9625 | Val Loss: 1.0421 Acc: 0.8353 | LR: 9.55e-05
Epoch 37/200 | Train Loss: 0.8192 Acc: 0.9625 | Val Loss: 1.0538 Acc: 0.8118 | LR: 9.51e-05
Epoch 38/200 | Train Loss: 0.7936 Acc: 0.9700 | Val Loss: 1.0662 Acc: 0.8118 | LR: 9.47e-05
Epoch 39/200 | Train Loss: 0.7997 Acc: 0.9650 | Val Loss: 1.0716 Acc: 0.8118 | LR: 9.44e-05
Epoch 40/200 | Train Loss: 0.7972 Acc: 0.9725 | Val Loss: 1.0639 Acc: 0.8235 | LR: 9.40e-05
Epoch 41/200 | Train Loss: 0.7846 Acc: 0.9675 | Val Loss: 1.0591 Acc: 0.8118 | LR: 9.36e-05
Epoch 42/200 | Train Loss: 0.7961 Acc: 0.9725 | Val Loss: 1.0582 Acc: 0.8235 | LR: 9.32e-05
Epoch 43/200 | Train Loss: 0.8114 Acc: 0.9675 | Val Loss: 1.0560 Acc: 0.8235 | LR: 9.27e-05
Epoch 44/200 | Train Loss: 0.7839 Acc: 0.9725 | Val Loss: 1.0518 Acc: 0.8353 | LR:

9.23e-05

Epoch 45/200 | Train Loss: 0.7767 Acc: 0.9800 | Val Loss: 1.0476 Acc: 0.8353 | LR: 9.19e-05

Epoch 46/200 | Train Loss: 0.7799 Acc: 0.9725 | Val Loss: 1.0454 Acc: 0.8353 | LR: 9.14e-05

Epoch 47/200 | Train Loss: 0.7819 Acc: 0.9775 | Val Loss: 1.0514 Acc: 0.8000 | LR: 9.09e-05

Epoch 48/200 | Train Loss: 0.7844 Acc: 0.9650 | Val Loss: 1.0504 Acc: 0.8000 | LR: 9.05e-05

Epoch 49/200 | Train Loss: 0.7862 Acc: 0.9725 | Val Loss: 1.0500 Acc: 0.8118 | LR: 9.00e-05

Epoch 50/200 | Train Loss: 0.7697 Acc: 0.9675 | Val Loss: 1.0522 Acc: 0.8353 | LR: 8.95e-05

Epoch 51/200 | Train Loss: 0.7815 Acc: 0.9800 | Val Loss: 1.0483 Acc: 0.8588 | LR: 8.89e-05

Epoch 52/200 | Train Loss: 0.7787 Acc: 0.9750 | Val Loss: 1.0565 Acc: 0.8471 | LR: 8.84e-05

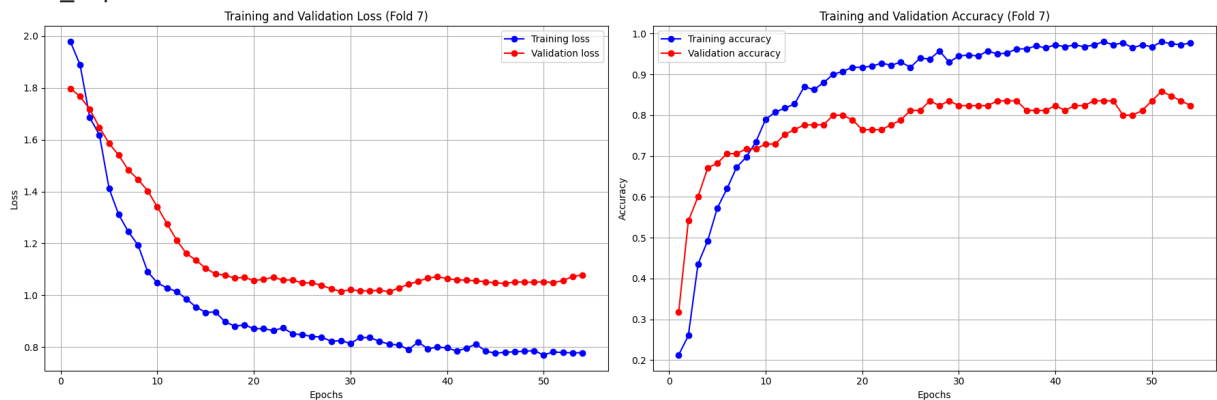
Epoch 53/200 | Train Loss: 0.7778 Acc: 0.9725 | Val Loss: 1.0722 Acc: 0.8353 | LR: 8.79e-05

Epoch 54/200 | Train Loss: 0.7783 Acc: 0.9775 | Val Loss: 1.0783 Acc: 0.8235 | LR: 8.73e-05

Early stopping triggered.

Training complete in 7.3 minutes. Best Val Loss: 1.0135

Best model for fold saved to: new_results_Multimodal_VGG16_Concat_KFold\best_model_fold_7.pth



Saved training history plot to: new_results_Multimodal_VGG16_Concat_KFold\training_history_fold_7.png

=====

OVERALL PERFORMANCE (FOLD 7)

=====

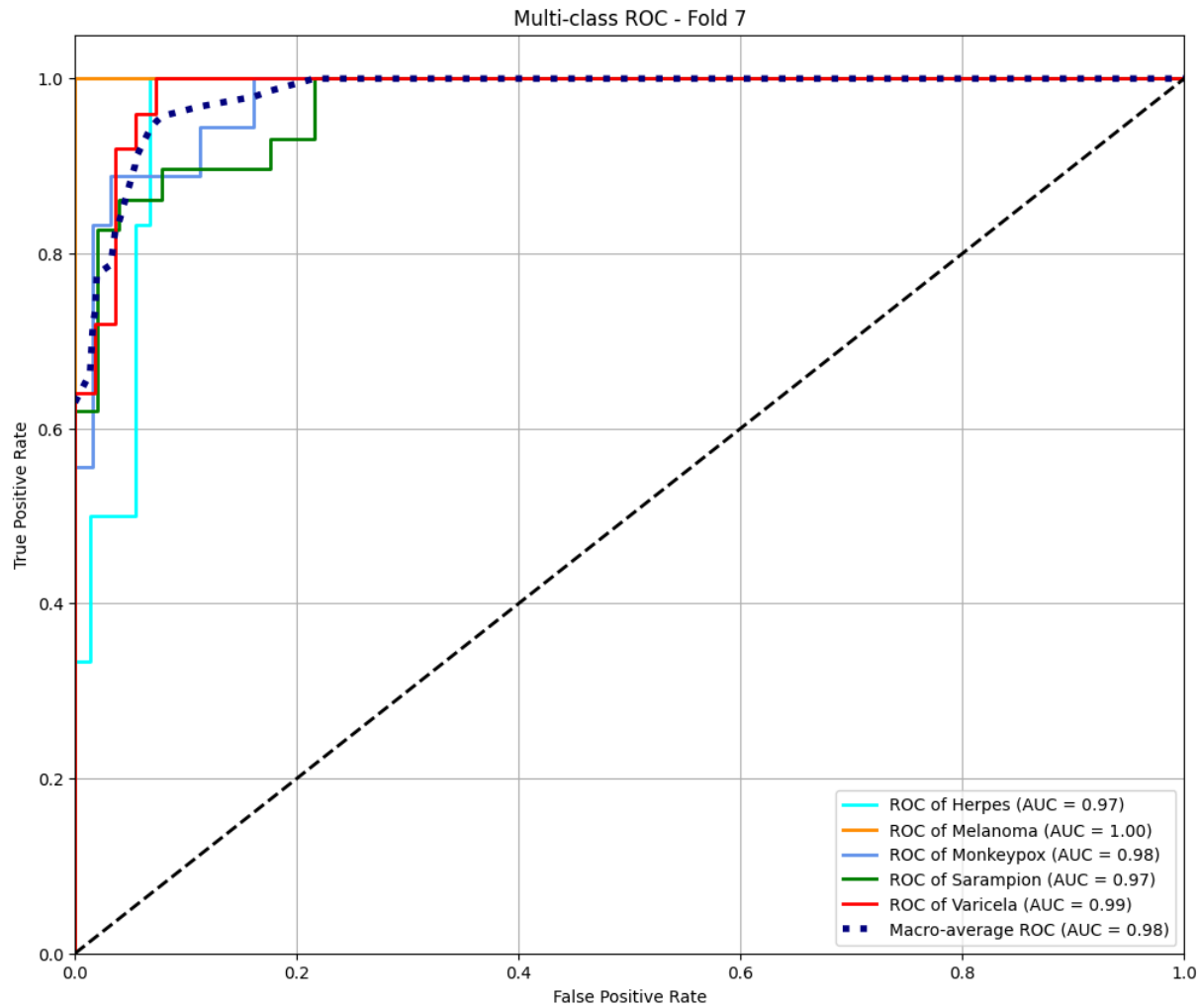
	precision	recall	f1-score	support
Herpes	0.57	0.67	0.62	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.80	0.89	0.84	18
Sarampion	0.96	0.83	0.89	29
Varicela	0.88	0.92	0.90	25
accuracy			0.86	80
macro avg	0.84	0.86	0.85	80
weighted avg	0.87	0.86	0.86	80

--- Overall Scalar Metrics ---

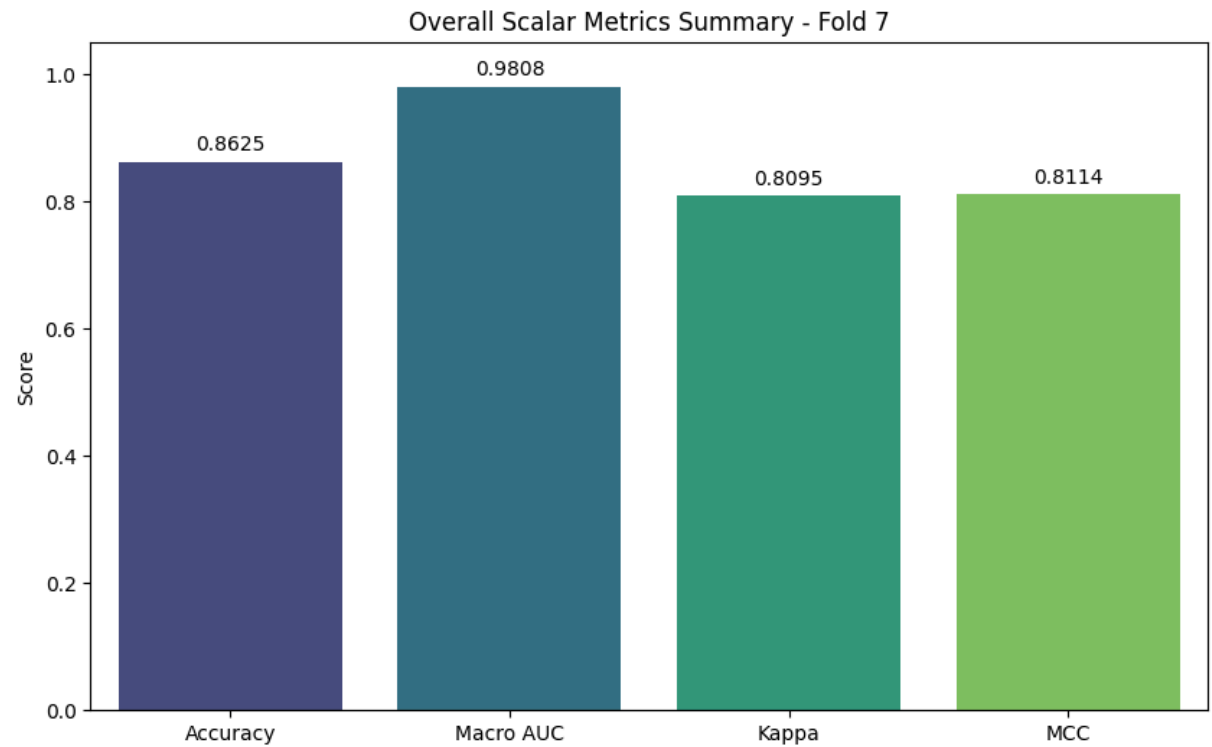
Accuracy: 0.8625
Macro ROC AUC: 0.9808
Cohen's Kappa: 0.8095
Matthews CorrCoef: 0.8114



Saved normalized confusion matrix to: new_results_Multimodal_VGG16_Concat_KFold\confusion_matrix_fold_7.png



Saved ROC curves plot to: new_results_Multimodal_VGG16_Concat_KFold\roc_curves_fold_7.png



Saved scalar metrics plot to: new_results_Multimodal_VGG16_Concat_KFold\scalar_metrics_fold_7.png

=====

PERFORMANCE BY GENDER

=====

--- Subgroup: female ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.75	1.00	0.86	3
Sarampion	1.00	1.00	1.00	15
Varicela	1.00	0.92	0.96	12
accuracy			0.97	30
macro avg	0.55	0.58	0.56	30
weighted avg	0.97	0.97	0.97	30

--- Subgroup: male ---

	precision	recall	f1-score	support
Herpes	0.57	0.67	0.62	6
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.81	0.87	0.84	15
Sarampion	0.90	0.64	0.75	14
Varicela	0.80	0.92	0.86	13
accuracy			0.80	50
macro avg	0.82	0.82	0.81	50
weighted avg	0.81	0.80	0.80	50

=====

PERFORMANCE BY AGE

=====

--- Subgroup: 0-17 ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.67	1.00	0.80	2
Sarampion	1.00	0.67	0.80	3
Varicela	0.80	0.80	0.80	5
accuracy			0.73	11
macro avg	0.49	0.49	0.48	11
weighted avg	0.76	0.73	0.73	11

--- Subgroup: 18-59 ---

	precision	recall	f1-score	support
Herpes	0.67	0.80	0.73	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.81	0.87	0.84	15
Sarampion	0.96	0.85	0.90	26
Varicela	0.90	0.95	0.93	20
accuracy			0.88	68
macro avg	0.87	0.89	0.88	68
weighted avg	0.89	0.88	0.88	68

--- Subgroup: 60+ ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	1
Sarampion	0.00	0.00	0.00	0
Varicela	0.00	0.00	0.00	0
accuracy			1.00	1
macro avg	0.20	0.20	0.20	1
weighted avg	1.00	1.00	1.00	1

=====

PERFORMANCE BY SKIN TONE

=====

--- Subgroup: asian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.80	1.00	0.89	4
Sarampion	1.00	0.80	0.89	5
Varicela	0.75	0.75	0.75	4
accuracy			0.85	13
macro avg	0.51	0.51	0.51	13
weighted avg	0.86	0.85	0.85	13

--- Subgroup: black ---

	precision	recall	f1-score	support
Herpes	0.33	1.00	0.50	1
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.80	0.80	0.80	5
Sarampion	0.00	0.00	0.00	2
Varicela	1.00	1.00	1.00	1
accuracy			0.67	9

macro avg	0.43	0.56	0.46	9
weighted avg	0.59	0.67	0.61	9

--- Subgroup: indian ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	4
Sarampion	1.00	1.00	1.00	1
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Subgroup: latino hispanic ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	1.00	1.00	1.00	2
Sarampion	1.00	1.00	1.00	3
Varicela	0.00	0.00	0.00	0
accuracy			1.00	5
macro avg	0.40	0.40	0.40	5
weighted avg	1.00	1.00	1.00	5

--- Subgroup: middle eastern ---

	precision	recall	f1-score	support
Herpes	0.00	0.00	0.00	0
Melanoma	0.00	0.00	0.00	0
Monkeypox	0.00	0.00	0.00	0
Sarampion	1.00	0.50	0.67	2
Varicela	0.00	0.00	0.00	0
accuracy			0.50	2
macro avg	0.20	0.10	0.13	2
weighted avg	1.00	0.50	0.67	2

--- Subgroup: white ---

	precision	recall	f1-score	support
Herpes	0.75	0.60	0.67	5
Melanoma	1.00	1.00	1.00	2
Monkeypox	0.67	0.67	0.67	3
Sarampion	0.94	0.94	0.94	16
Varicela	0.90	0.95	0.93	20
accuracy			0.89	46

macro avg	0.85	0.83	0.84	46
weighted avg	0.89	0.89	0.89	46

```
=====
FAIRNESS SUMMARY (ALL GROUPS) | WGA: 0.0000 EOD: 0.7833
Worst pair: 0-17|male|asian vs 18-59|male|white
=====
```

```
=====
FAIRNESS SUMMARY (FILTERED, n>=15) | WGA: 0.8333 EOD: 0.6056
Worst pair: 18-59|female|white vs 18-59|male|white
Groups: ['18-59|female|white', '18-59|male|white']
=====
```

```
=====
====
Cross-validated over 7 folds
=====
```

```
====
Accuracy          : 0.860 ± 0.028
Roc Auc Macro     : 0.961 ± 0.035
Kappa             : 0.808 ± 0.039
Mcc               : 0.810 ± 0.039
Wga All           : 0.095 ± 0.252
Eod All           : 0.856 ± 0.085
Wga Filtered      : 0.820 ± 0.115
Eod Filtered      : 0.444 ± 0.248
```

```
--- POOLED OVERALL ACCURACY BY GENDER (n>=15) ---
```

Group	Acc	Count
male	0.828	355
female	0.914	210

```
--- POOLED OVERALL ACCURACY BY AGE (n>=15) ---
```

Group	Acc	Count
18-59	0.872	501
0-17	0.754	61

```
--- POOLED OVERALL ACCURACY BY SKIN TONE (n>=15) ---
```

Group	Acc	Count
white	0.870	338
asian	0.829	70
indian	0.820	50
latino hispanic	0.940	50
black	0.842	38
middle eastern	0.737	19

```
--- POOLED OVERALL ACCURACY BY COMPOSITE (n>=15) ---
```

Group	Acc	Count
18-59 male white	0.819	166
18-59 female white	0.960	126
18-59 male asian	0.846	39
18-59 male indian	0.853	34
18-59 male black	0.867	30
0-17 male white	0.833	30

18-59 female asian	0.885	26
18-59 female latino hispanic	0.920	25
18-59 male latino hispanic	0.957	23
18-59 male middle eastern	0.722	18
0-17 female white	0.733	15

Results, models, and plots saved to 'new_results_Multimodal_VGG16_Concat_KFold'

In []: